Computers & Security 173 (2027) 105206

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Detecting and preventing lateral movement and privilege escalation attacks within microsoft active directory

Tegha Hamza Akwo [a,b], Dekai Zhang [a,b], Lu Yu [a,c], Bing Zhang [a,b,*]

[a] School of Artificial Intelligence (School of Software), Yanshan University, Qinhuangdao, 066004, China
[b] Key Laboratory of Software Engineering of Hebei Province, Yanshan University, Qinhuangdao, 066004, China
[c] Hebei Port Group Datalink Technology (Xiongan) Co., Ltd, China

ARTICLE INFO

Dataset link: https://github.com/TeghaHamza/VACIR-Engine-Code-/releases/tag/V1.0

Keywords:
Microsoft AD
Lateral movement
Privilege escalation
Attack path discovery
Automated security analysis
Domain compromise
VACIR

ABSTRACT

Enterprise Active Directory (AD) systems inherently struggle to maintain security boundaries across complex identity infrastructures. Misconfigurations like overly permissive access control lists (ACLs) and insecure delegation frequently enable lateral movement and privilege escalation. Existing tools successfully enumerate these attack paths, but their subsequent remediation process remains highly manual. Current methods also lack formal mathematical guarantees that proposed security fixes are minimal or operationally safe from service disruption. We introduce VACIR (Vulnerability Aware Contextual Intervention and Remediation), an AD remediation framework that maps SharpHound data and CVE intelligence into a weighted directed graph. VACIR uniquely applies Integer Linear Programming (ILP) and Satisfiability Modulo Theories (SMT) to compute optimal, cost-effective hardening actions. The framework correlates AD attributes, ACLs, and session states to identify precise intervention points under strict service preservation constraints. We rigorously evaluated VACIR on a controlled enterprise domain and three public AD datasets against BloodHound Analyzer and CoZure. Empirical results demonstrate high detection precision with sub-second runtimes, vastly improving coverage over existing baseline tools. These findings validate the practicality and scalability of optimization-driven, attack-path-centric defense in modern enterprise AD environments.

1. Introduction

Microsoft AD (Microsoft Corporation, 2025; Nguyen and Nguyen, 2023) remains the foundational identity and access management platform for Windows domain environments and continues to be a central component of enterprise security architectures. Its wide adoption and deep integration into corporate authentication, authorization, and policy-enforcement workflows make it an attractive and frequent target for adversaries seeking to escalate privileges (Carson, 2022) or compromise critical assets.

Although cloud-based identity platforms such as Azure AD (Quest, 2026) have emerged to extend these capabilities, many of the structural weaknesses and misconfiguration challenges historically associated with on-premises AD persist across both environments.

Numerous high-profile intrusions have shown that attackers rarely rely on a single exploit; instead, they chain together software vulnerabilities (WindowsForum, 2025) (e.g., CVEs), misconfigured delegation, weak service accounts, and overly permissive access control lists (ACLs) to escalate from a low-privilege foothold to full Domain Admin rights. These weaknesses frequently originate from complex network and identity configurations where security principles such as least privilege and tiered administration are difficult to implement and maintain in practice (Nguyen and Nguyen, 2023).

To support defenders, the community has developed sophisticated tools (Splunk, 2025; Microsoft, 2025; Felicite et al., 2024) that automate attack path discovery. A common approach is to extract a logical model of the AD environment and enumerate relationships between objects to identify privilege escalation opportunities. Tools such as BloodHound (SpecterOps, 2025a) and SharpHound (SpecterOps, 2025b) construct an attack graph in which a directed edge from node A to node B represents that an attacker can move from A to B via a specific primitive (Zhou et al., 2024). Notwithstanding their effectiveness in mapping attack paths, these existing tools remain mainly analytical rather than solution-oriented. The main fundamental limitations that arise are outlined as follows.

Existing work relies largely on manual processes. Existing tools such as BloodHound, SharpHound, and related graph-based enumeration

* Corresponding author.
E-mail addresses: teghahamzaakwo@stumail.ysu.edu.cn (T.H. Akwo), linkzhangdekai@gmail.com (D. Zhang), yulu@portqhd.com (L. Yu), bingzhang@ysu.edu.cn (B. Zhang).

https://doi.org/10.1016/j.cose.2026.105206
Received 13 July 2026; Received in revised form 15 September 2026; Accepted 10 October 2026
Available online 29 October 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

frameworks (Guo et al., 2022; Herranz-Oliveros et al., 2024) produce detailed visualizations of attack paths and privilege relationships within AD environments. However, the process of interpreting these outputs and determining appropriate remediation actions remains a predominantly manual, expert-driven task. Security analysts must individually examine each discovered path, assess its operational relevance, and manually formulate a corrective action—a process that is inherently time-consuming, error-prone, poorly scalable, and demanding of significant analyst expertise. Prior studies have documented that enterprise AD environments can contain thousands of exploitable relationships (Kohler, 2022), and manually triaging such volumes of findings often results in analyst fatigue, delayed remediation, and incomplete coverage. Nguyen et al. (2025) similarly observed that continuous monitoring and attack-path validation in AD environments demand automated frameworks, as manual inspection alone cannot keep pace with the rate at which new misconfigurations and vulnerabilities are introduced through routine administrative operations. Furthermore, incident response reports indicate that attackers can achieve domain compromise within minutes to days once an initial foothold is established (Kapko, 2026), whereas manual remediation workflows typically operate on timescales of weeks to months (Tenable, 2026), creating a critical temporal asymmetry between adversarial exploitation and defensive response.

The current work provides no assurance that the chosen mitigations are optimal or operationally secure. Even when remediation actions are identified, existing approaches provide no formal guarantee that the selected set of interventions is either optimal (e.g., achieves the maximum security improvement for a given cost) or operationally safe (e.g., does not inadvertently disrupt legitimate services or introduce Denial-of-Service conditions). Current tools such as BloodHound Analyzer present findings as isolated alerts without contextualizing them within a holistic remediation strategy. As a consequence, defenders frequently apply ad-hoc fixes; for example, removing a single ACL (Microsoft, 2026) entry or disabling one service account without evaluating whether the intervention addresses the root cause or merely displaces the attack path to an alternative route (SpecterOps, 2025a). Moric et al. (2024) demonstrated through penetration testing that AD configuration vulnerabilities often exist in interconnected clusters, where remediating one misconfiguration without addressing related weaknesses leaves residual attack paths intact. Similarly, Cordis et al. (2023) noted that Kerberos vulnerability mitigation in AD requires coordinated changes across encryption policies, service principal configurations, and delegation settings; isolated fixes targeting only one aspect may break service authentication while failing to fully neutralize the threat. The absence of a principled mechanism for evaluating the joint effect of multiple remediation actions means that defenders cannot distinguish between a minimal, high-impact intervention set and a larger, potentially disruptive set of changes that may degrade system availability. This limitation is particularly acute in enterprise environments where AD underpins critical business services including email, file sharing, enterprise resource planning, and single sign-on authentication.

Apart from the two points listed above, another limitation is the lack of a clear mathematical method for finding the best way to disrupt attack paths while still keeping the system working properly. The most fundamental gap in the existing literature is the lack of a rigorous, optimization-based methodology for computing the minimal set of defensive actions that maximally disrupt adversarial attack paths while preserving legitimate operational functionality. Current approaches to AD hardening rely on heuristic prioritization (e.g., addressing "high-severity" findings first) or compliance-driven checklists (e.g., CIS benchmarks, Microsoft security baselines) that do not account for the graph-theoretic structure of the attack surface. Felicite et al. (2026) proposed detection-oriented approaches for specific attack primitives such as Pass-the-Hash but did not address the broader problem of selecting an optimal combination of mitigations across heterogeneous vulnerability classes. Meghana et al. (2025) developed log correlation methods for detecting enumeration and post-enumeration attacks on AD but similarly focused on detection rather than remediation planning. The CoZure framework introduced by Chowdhury and Nguyen (Nguyen and Nguyen, 2023) explored context-free grammar-based approaches for discovering new lateral movement primitives in Azure AD; however, it did not formulate remediation as an optimization problem and, as demonstrated in our evaluation (Detailed analysis in Table 2 of Section 4.4.1), did not produce actionable detection outputs on the evaluated datasets. Zhou et al. (2024) proposed time-aware subgraph classification for lateral movement detection but operated exclusively in the detection domain without extending to remediation planning. In mathematical terms, the problem of selecting a minimum-cost subset of edge removals from an attack graph such that all paths from a source set to a target set are severed, subject to constraints preserving operationally critical edges, is recognizable as shortest-path network interdiction. This constitutes a constrained minimum cut problem, which is naturally formulated as an Integer Linear Programming (ILP) or Satisfiability Modulo Theories (SMT) instance. To the best of our knowledge, no prior work has applied such formal optimization techniques to AD attack path remediation.

To address these challenges, this paper proposes a model that moves from attack path discovery to attack-path-centric remediation by developing new methods to compute optimized defensive actions for AD. To this end, we introduce VACIR (Vulnerability Aware Contextual Intervention and Remediation), a remediation co-pilot for AD. Rather than treating the BloodHound graph as a static visualization, VACIR treats it as a structured input to a formal optimization engine. We observe that most AD compromise chains can be modeled as sequences of local configuration conditions such as delegation flags, ACL edges, and group memberships that combine to push the system into high-risk states such as domain takeover or domain-wide Denial of Service (DoS). Security documentation and prior attack graphs already encode which combinations of edges enable which escalation steps. However, deciding which connections should be removed or changed in order to minimize the attack surface while causing the least disruption to operations still requires expert knowledge and human decision-making. Our primary contributions are as follows:

- We introduce VACIR, the first remediation-planning framework that combines attack-graph analysis with configuration-state modeling and single-objective optimization to generate actionable, low-impact AD hardening strategies.
- We develop an automated orchestration pipeline capable of applying baseline configurations, deploying optimized remediations, and capturing experimental snapshots for measurable before-and-after comparison.
- We design a comprehensive evaluation methodology using controlled, repeatable red-team exercises to measure improvements in time-to-compromise, number of viable attack paths, and system stability across remediation strategies.
- We present experimental evidence showing that VACIR reduces 159 verified exploitable attack paths in a controlled enterprise AD dataset (including 12 lateral-movement paths and 128 privilege-escalation edges) while maintaining Precision = 1.00, thereby minimizing disruptive false-positive remediation actions. Across four datasets, VACIR achieves an average F1-score of 0.84 with sub-second execution time (≤ 0.29 s), outperforming conventional graph-analysis baselines.

The rest of this paper is organized as follows. Section 2 which is about the background on lateral movement and privilege escalation within AD, along with a review of related work. Section 3 detailing the proposed VACIR methodology, outlining the data acquisition process,

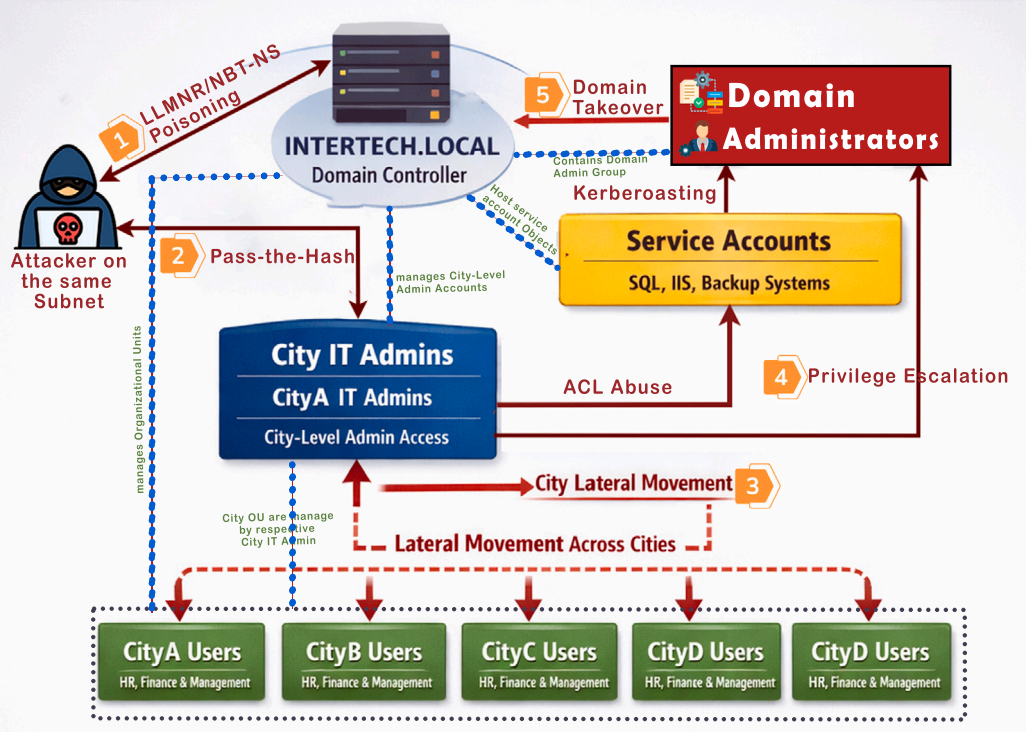


Fig. 1. Leveraging Lateral Movement and Privilege Escalation in Active Directory.

formal graph modeling, and the optimization-based analysis engine. Section 4 presents the experimental environment, case studies, and an empirical evaluation comparing our framework against existing state-of-the-art baselines. Section 5 discusses the practical advantages, current limitations, and avenues for future research. Finally, Section 6 concludes the paper.

2. Background and related work

To provide context for the proposed VACIR framework, this section reviews the concepts and existing literature related to AD security. Section 2.1 outlines the mechanics of lateral movement and privilege escalation, detailing how adversaries chain misconfigurations to compromise enterprise domains. Section 2.2 examines current manual and heuristic-based approaches to attack path identification, while Section 2.3 highlights the limitations of prior optimization models, establishing the necessity for an automated, context-aware defensive solution.

2.1. Lateral movement and privilege escalation in microsoft active directory

Microsoft AD environments are highly susceptible to identity-based attack paths that enable adversaries to escalate privileges and move laterally across the network. These attack paths typically begin when an attacker gains initial access to a compromised host or low-privilege account within the domain. Once inside the network, the attacker performs a sequence of operations that gradually increase their privileges while expanding their reach across domain resources. This process is commonly referred to as lateral movement and privilege escalation, and it often culminates in the compromise of the Domain Controller, resulting in full domain takeover.

Fig. 1 illustrates a typical attack progression within an AD environment. In Stage 1, the attacker, positioned on the same subnet, exploits legacy name resolution weaknesses through Link-Local Multicast Name Resolution-NetBIOS Name Service (LLMNR/NBT-NS) poisoning to intercept NT LAN Manager version 2 (NTLMv2) authentication hashes from domain-joined workstations. In Stage 2, the captured hashes are leveraged via Pass-the-Hash attacks to authenticate as intermediate administrative accounts, such as City IT Admins, who manage specific organizational units within the domain. Stage 3 involves lateral movement, in which the attacker exploits the compromised city-level administrative privileges to traverse multiple organizational units (example CityA through CityD) via active Remote Desktop Protocol/Server Message Block(RDP/SMB) sessions, progressively expanding access to additional hosts and credential caches. In Stage 4, privilege escalation is achieved through two complementary vectors: exploiting misconfigured ACLs (example, GenericWrite permissions over high-value groups) and performing Kerberoasting attacks against Service Accounts (e.g., SQL, IIS, Backup Systems) configured with extractable SPNs, enabling offline credential cracking and access to accounts with elevated domain-wide privileges. Finally, Stage 5 culminates in domain takeover, where the attacker obtains Domain Administrator membership and gains unrestricted control over the Domain Controller (INTERTECH.LOCAL), enabling persistent access, policy manipulation, and complete compromise of the enterprise identity infrastructure.

The critical challenge in securing AD environments lies in identifying and eliminating these potential lateral movement paths before they can be exploited. Modern attack-path analysis tools such as BloodHound model the relationships between users, computers, groups, and permissions to identify privilege escalation routes within the domain. However, these tools rely primarily on predefined attack primitives and known misconfiguration patterns. In practice, attackers may exploit previously unseen combinations of permissions, services, and authentication mechanisms to construct new attack paths. Consequently, advanced detection approaches are required to analyze complex AD graphs and identify emerging lateral movement strategies that could lead to domain compromise.

2.2. Manual identification of lateral movement and privilege escalation paths in active directory

In recent years, security researchers, red team operators, and incident responders have traditionally identified lateral movement and

Table 1
Comparison summary.

Method	Primary objective	Input	Results	Remediation	Optimization	Safety	Output
BloodHound SharpHound (SpecterOps, 2025a)	Attack-path discovery via AD graphs	AD rel./premises data by SharpHound	Query-driven graph analysis	Manual expert-driven	Heuristic and ad-hoc	May cause DoS and disruption	Graph plus manual triage
CoZure (Nguyen and Nguyen, 2023)	Discover new lateral movement premises in Azure AD	Azure AD env. model	Grammar based enum.	No optimization	None formulated	Not Stated	New Azure AD primitives
Tech.-spec. detect. Felicite et al. (2024) (Zhou et al., 2024) (Meghana et al., 2024)	Detect specific attacks (Pass-the-Hash etc.)	Logs and domain signals	Detection-only methods	Detection focus only	None	Not stated	Alerts and suspicious behavior
VACIR	Attack-path hardening remediation planning	BloodHound output + vuln. misconfig knowledge + CVE maps	Weighted ILP and SMT optimization modeling	Prioritized ACL, Kerberos and delegation hardening set	ILP+SMT minimal interventions	Constraints avoid unsafe and service-outage actions	Remediation plan + lateral - movement report

privilege escalation paths in Microsoft AD through manual inspection of domain users, groups, computers, sessions, and permission assignments. This process involves examining domain relationships object by object in order to determine whether a compromised account can move laterally to another host or escalate to a more privileged identity. In practice, however, such an approach is often slow, labor-intensive, and error-prone, particularly in enterprise environments containing hundreds or thousands of highly interconnected objects.

A major limitation of this manual methodology is that the search and correlation process requires substantial time, while adversaries can exploit the same environment much more rapidly. For example, prior studies and incident reports have shown that attackers may achieve domain compromise within minutes if they remain undetected and no mitigation strategy is promptly applied (Horizon3, 2025). Other reports further indicate that a domain controller can be fully compromised within a matter of days once an attacker establishes a foothold and progressively chains available privileges (cyber.gov.au/, 2024). This imbalance motivates the need for automated approaches capable of rapidly discovering, correlating, and prioritizing lateral movement and privilege escalation paths before an adversary reaches high-value assets such as the Domain Controller.

To address the limitations of manual evaluation, a notable body of work has successfully applied formal optimization to attack-graph defense. Beyond the Active Directory literature, the minimum cost hardening line beginning with Noel et al. (2003), Wang et al. (2006) established foundational models for severing attack paths. This was further expanded by multi-objective treatments in Dewri et al. (2007) and the "Heat-Ray" approach by Dunagan et al. (2009). More recently, formal optimization concepts have been extended directly to Active Directory. Specifically, Guo et al. (2022) introduced defender-side fixed-parameter algorithms for AD attack graphs, while subsequent research by Guo et al. (2023) addressed the budgeted edge-blocking problem via mixed-integer programming, incorporating critical operational constraints to maintain normal functional access and evaluating the methodology on large synthetic graphs with tens of thousands of nodes. Related contributions include (Goel et al., 2022), which established computational hardness results for the same problem.

While this body of prior optimization work has made significant theoretical strides, it predominantly operates on abstracted graphs with uniform edge semantics fitting the classical paradigm of shortest path network interdiction. The primary novelty of VACIR lies in its practical application: rather than generating abstracted edge blocking recommendations, our cost model is grounded in concrete SharpHound attributes, and our output maps directly to executable, context-aware remediation.

2.3. Comparison of related work with VACIR

In order to clearly highlight how VACIR differs from existing approaches, Table 1 summarizes representative tools and research frameworks discussed in this chapter and compares them across input sources, supported attack categories, remediation capability, and the presence of formal optimization and operational-safety constraints. This comparison highlights that most prior work focuses on attack path discovery or detection, while VACIR explicitly targets attack-path-centric remediation by mapping SharpHound-derived relationships into a weighted security state graph and computing constrained intervention actions via ILP/SMT.

3. Methodology

The VACIR framework consists of four primary modules: Data Capture and Threat Model, Graph Model Formulation, Vulnerability Management, and Evaluation. Fig. 2 provides a detailed breakdown of these modules, which are elaborated in Section 3.1.1.

3.1. Data capture and threat model

Section 3.1 is organized to describe how the experimental evidence is obtained and specify the adversarial assumptions under which that evidence is interpreted. Section 3.1.1 presents the adversary-centric data acquisition workflow that produces the raw enumeration artifact D_{raw} from an initial zero-knowledge state S_0. Section 3.1.2 then formalizes the corresponding threat model and system context that is, the attacker capabilities, objectives, and defender assumptions ensuring that the captured dataset and subsequent analysis are evaluated under a clearly defined and reproducible security setting.

3.1.1. The data capture model

The Data Capture and Preprocessing Module is responsible for generating a high fidelity, adversary-centric representation of the AD environment. Unlike administrative exports that offer a static, privileged view, this module emulates an external attacker by employing a controlled workflow from a black-box perspective, beginning with zero knowledge of the target network's internal structure. This adversarial approach ensures the resulting dataset accurately reflects a real-world attack progression. The process is formalized as a composite function,

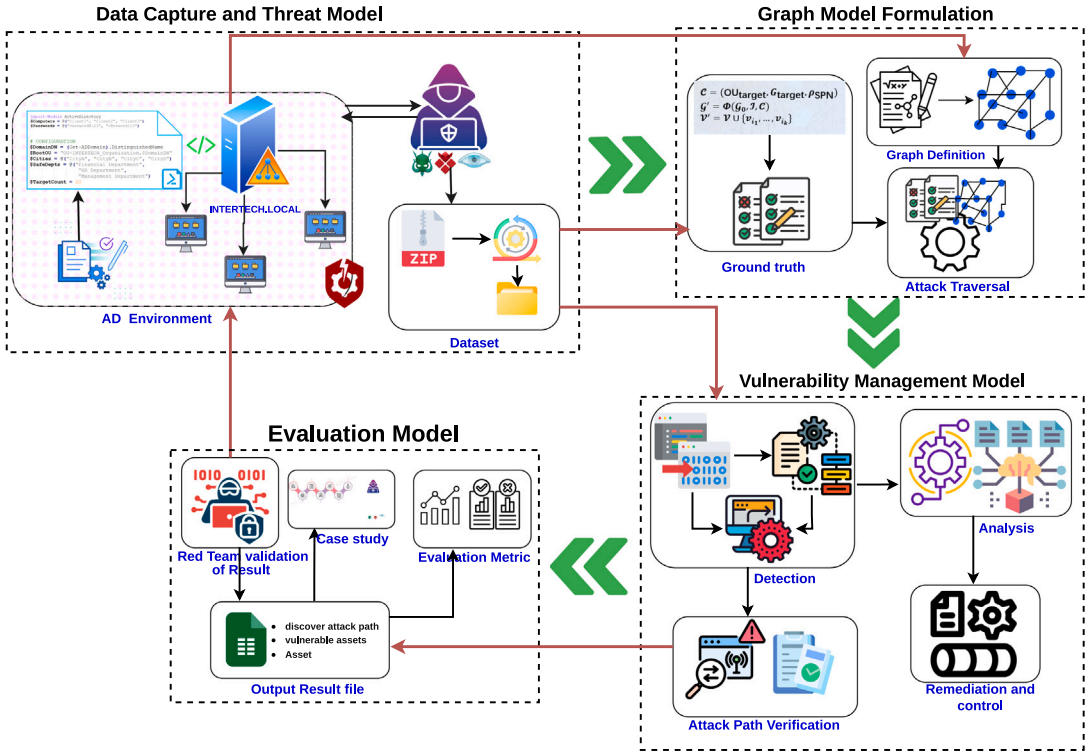


Fig. 2. VACIR workflow including four modules: Data capture and threat model, graph model formulation, vulnerability management model, and evaluation model. Data capture and graph model formulation define the mathematical relationship between the AD environment and how the dataset is captured; the vulnerability management model detects the various privilege escalation and lateral movement paths in the dataset; and the evaluation model compares and validates results.

F, that transforms an initial zero-knowledge state, S_0, into a fully enumerated attack graph, $G = (V, E)$, defined as follows:

$$G = (V, E) = F(S_0) = E \circ X \circ A \circ K \circ C \circ R(S_0) \tag{1}$$

Where:

- V is the finite set of nodes representing AD objects. Typical elements of V include: user accounts, computer objects, security groups, Group Policy Objects (GPOs), Organizational Units (OUs), containers, domain objects, and trust relationships.
- E is a set of directed edges and an edge include: MemberOf, GenericAll, GenericWrite, WriteOwner, WriteDACL, Owns, AdminTo, CanRDP, CanPSRemote, ExecuteDCOM, HasSession, GetChanges, GetChangesAll (DCSync rights), AllowedToDelegate, AllowedToAct, TrustedToAuthForDelegation, ForceChangePassword, AddMember, AddSelf, and other delegation or ACL-based primitives.

The composite function F is itself the composition of the attacker's sequential operations:

- Reconnaissance (R): Network scanning (using Nmap) to identify live hosts, services, and the AD Domain Controller.
- Credential Capture (C): Network poisoning (using LLMNR/NBT-NS spoofing with Responder) to intercept authentication hashes.
- Credential Cracking (K): Offline dictionary attacks (this done using Hashcat) to recover plaintext passwords from captured hashes.
- Authenticated Access (A): Establishing a remote shell (with Evil-WinRM) on a target system with recovered credentials.
- Privilege Escalation (X): Achieving Domain Admin-level access.
- Enumeration (E): Post-compromise data collection (achievable using SharpHound) to extract comprehensive details on AD objects, relationships, sessions, and permissions.

Successful execution of $F(S_0)$ confirms Domain Administrator compromise because the resulting graph G contains at least one reachable path from the attacker's initial foothold to a Domain Administrator principal.

The practical implementation of this conceptual model is detailed in Algorithm 1. The DATACAPTURE procedure outlines the sequential execution of the toolchain, translating the abstract functions of Eq. (1) into concrete, operational steps.

As shown in Algorithm 1, the process begins with Phase 1, Reconnaissance and Service Discovery (lines 3–4), which directly corresponds to the function R. Phase 2, Credential Interception and Recovery (lines 6–7), combines the credential capture (C) and cracking (K) operations to obtain valid credentials. A critical check (line 8) ensures the process terminates if authentication fails. Following a successful credential recovery, Phase 3, Authenticated Access and Enumeration (lines 12–13), executes the authenticated access (A) and comprehensive data enumeration (E) steps, yielding the raw data archive, D_{raw}. Finally, Phase 4, Exfiltration (line 15), ensures this archive is transferred to the analyst machine. The primary output of this module, the raw data archive D_{raw}, is subsequently parsed to construct the in-memory graph representation, which serves as the foundational data structure for the Analysis Engine.

3.1.2. Threat model

Our framework is designed to operate within a representative enterprise Microsoft AD environment. Guided by this objective, the network layout and data acquisition process were carefully designed to emulate the structure and security relationships of a real-world organizational

Algorithm 1 Active Directory Data Capture

Input: Target Subnet S_{target}, Password Dictionary W
Output: Raw Active Directory Enumeration Archive D_{raw}

procedure DataCapture(S_{target}, W)
  Phase 1: Reconnaissance and Service Discovery
  $Hosts \leftarrow NmapScan(S_{target}, ports = \{88, 139, 389, 445\})$
  $DC \leftarrow IdentifyDomainController(Hosts)$
  Phase 2: Credential Interception and Recovery
  $H_{NTLMv2} \leftarrow ResponderPoisoning(LLMNR, NBT-NS)$
  $Creds_{valid} \leftarrow HashcatOfflineCrack(H_{NTLMv2}, W)$
  if $Creds_{valid} == \emptyset$ **then**
    return *Error: AuthenticationFailed*
  end if
  Phase 3: Authenticated Access and Enumeration
  $Session \leftarrow EvilWinRM(DC, Creds_{valid})$
  $D_{raw} \leftarrow SharpHoundCollect(Session)$
  Phase 4: Exfiltration
  $DownloadToAnalystMachine(D_{raw})$
  return D_{raw}
end procedure

domain. Fig. 4 of Section 4.1 illustrates the network topology and overall evaluation setup used in this study.

VACIR functions as a multi-stage pipeline that ingests SharpHound (SpecterOps, 2025b) data and CVE (WindowsForum, 2025) vulnerability intelligence to build a normalized, directed enterprise graph. Within this model, nodes represent security principals and assets, while weighted edges signify the relative exploitability of privilege relationships and session based access. By evaluating these security relevant weights, the framework's optimization component calculates a prioritized set of intervention actions designed to maximally disrupt adversary routes to high-value targets. Finally, the system generates a comprehensive remediation report that details vulnerable users and systems alongside the specific attack techniques required to exploit them. The adversary is modeled under a black-box threat assumption, starting with no prior knowledge of the Active Directory (AD) environment and relying on standard enumeration/post-exploitation tools to map users, groups, hosts, trusts, sessions, and privileges. This reflects a realistic enterprise intrusion progression from limited initial visibility to an increasingly detailed understanding of domain structure. The study environment is the INTERTECH.local AD domain (Fig. 5 provides details, which can be seen in Section 4.2), organized with a Root Domain at the top. Beneath the root are two main components: WINSERVER (the Domain Controller) and the INTERTECH Organization Organizational Unit (OU). WINSERVER hosts AD DS and includes an SMB share (Backups) containing a deliberately exposed sensitive file (Credentials.txt) to simulate common misconfiguration-based lateral movement opportunities. The INTERTECH Organization OU is geographically segmented into City A–D, each further divided into departmental OUs (for example HR/FIN/Management) containing normal user accounts.

3.2. Graph model formulation

To systematically model and counter the adversary behaviors outlined in Section 3.1, the AD environment is abstracted into a formal mathematical construct. This allows us to reason about security posture and attack viability with computational rigor. We model the AD infrastructure as a multi-relational, weighted, directed graph, termed the Security State Graph.

3.2.1. Security state graph definition

Building upon the Data Capture and Preprocessing Module (Section 3.1.1), the Security State Graph reflecting the enterprise AD structure (such as the compartmentalized INTERTECH.LOCAL domain) is formally defined from the output of the same composite function F introduced in Eq. (1). Specifically, the process transforms the initial zero-knowledge state S_0 into a fully enumerated Security State Graph $G = (V, E, R, A, W)$, defined as follows: To reflect the Tiered Administration model, the user set U is further stratified: $U = U_{DA} \cup U_{IT} \cup U_{Norm}$, corresponding to Domain Administrators (like the case of: j.praise), City-Level IT Admins, and Standard Users (example, a.birk).

- E the set of directed relationship types mapping to AD permissions and states (for example, E = MemberOf, GenericWrite, HasSession, CanRDP).
- $E \subseteq V \times R \times V$ is the set of directed edges, where an edge $e = (v_i, r, v_j)$ represents a relationship or actionable exploit primitive $r \in R$ from source v_i to target v_j.
- $A : V \rightarrow \mathbb{B}^k$ is a boolean attribute function defining vulnerability flags for a node (example, hasSPN = 1 for Kerberoasting, DontReqPreAuth = 1 for AS-REP Roasting).
- $W : E \rightarrow \mathbb{R}^+$, is a cost function assigning a real-valued weight to each edge, representing the operational difficulty, noise level, or time required for an attacker to traverse that edge. Lower weights indicate easily exploitable paths.

3.2.2. Ground truth graph definition

Our evaluation methodology relies on comparing detected paths against a known, deterministic baseline. This is achieved through a ground-truth provisioning function, Φ, which populates the initial graph G_0 with specific vulnerability subsets corresponding exactly to the empirical data categorized in Table 2 of Section 4.4.1 and Table 3 of Section 4.5.

Let P_{GT} be the set of Ground Truth attack edges injected into the environment. The function Φ executes transformations on G_0 to map the specific attack types observed in our dataset:

- Privilege Escalation (Group Membership Abuse):

For an initial low-privilege user $u \in U_{Norm}$ and a target administrative group $g_{target} \in G_{AD}$ (example, CityC IT Admins), if a misconfigured ACL exists, Φ injects an exploit edge:

$$\forall u \in \mathcal{U}_{Norm}, \text{ if } \mathrm{ACL}(u, g_{target}) \supseteq \mathrm{GenericWrite} \implies \mathcal{E} \leftarrow \mathcal{E} \cup \{(u, \mathrm{PrivEsc}, g_{target})\}$$

- Lateral Movement (Active Session Hijacking):

To model credential theft via Remote Desktop Protocol (RDP) or Server Message Block (SMB) active sessions, if a high-privilege user u_{admin} has an active session on a computer $c \in C$, and a compromised user u_{comp} has local admin rights over c:

$$\text{If } (c, \mathrm{HasSession}, u_{admin}) \in \mathcal{E} \wedge (u_{comp}, \mathrm{AdminTo}, c) \in \mathcal{E} \implies \mathcal{E} \leftarrow \mathcal{E} \cup \{(u_{comp}, \mathrm{LateralMove}, u_{admin})\}$$

- Offline Credential Cracking (Account Takeover):

The function Φ embeds Kerberoasting and AS-REP Roasting vulnerabilities by modifying node attributes and injecting low-cost attack edges to Domain Controllers (DC): If $\mathcal{A}(u).\mathrm{hasSPN} == 1 \vee \mathcal{A}(u).\mathrm{DontReqPreAuth} == 1 \implies \mathcal{E} \leftarrow \mathcal{E} \cup (v_{attacker}, \mathrm{OfflineCrack}, u)$ Where $W(v_{attacker}, \mathrm{OfflineCrack}, u) \leftarrow \omega_{low}$, representing the 64 credential theft vulnerabilities detected in the public and local datasets shown in Section 4.4.

3.2.3. Attack traversal and detection formulation

An attack is modeled as the discovery and traversal of a continuous path π in G from a low-privilege entry vertex v_{start} to a high-value target v_{target} An attack path is a sequence of edges $\pi = \langle e_1, e_2, \ldots, e_k \rangle$. The cumulative cost of an attack path is the sum of its edge weights:

$$\text{Cost}(\pi) = \sum_{i=1}^{k} \mathcal{W}(e_i) \tag{2}$$

Formally framed within the literature of shortest-path network interdiction and budgeted edge-blocking (Noel et al., 2003; Wang et al., 2006; Guo et al., 2023), the objective of our vulnerability management engine is to compute a minimum-cost hitting set that systematically disrupts these optimal adversarial paths without violating operational constraints.

The optimal attack strategy is defined as finding the path π^* that minimizes traversal cost:

$$\pi^* = \arg \min_{\pi \in \Pi(v_{\text{start}} \to v_{\text{target}})} \text{Cost}(\pi) \tag{3}$$

Where Π represents the universe of all viable paths between the source and target.

Detection Formulation:

Vulnerability Management model acts as a detection function $\Delta(G) \rightarrow P_{det}$, where P_{det} is the Detected Positive Set of exploitable paths. To achieve the optimization and automated remediation detailed later in the paper, VACIR categorizes P_{det} into mathematical subsets corresponding to our findings:

$$\mathcal{P}_{\text{det}} = \mathcal{P}_{LM} \cup \mathcal{P}_{PE} \cup \mathcal{P}_{AT} \tag{4}$$

Where P_{LM} represents Lateral Movement paths, P_{PE} represents Privilege Escalation, and P_{AT} represents Account Takeovers. To evaluate the engine's efficacy (as reported in Table 2 of Section 4.4.1), the detected set P_{det} is mathematically intersected with the known ground truth set P_{GT} established in Section 3.2.2. This allows us to formally define our evaluation metrics:

- True Positives (TP): $TP = |P_{det} \cap P_{GT}|$ (e.g., the 159 successfully identified vulnerabilities).
- False Positives (FP): $FP = |P_{det} \setminus P_{GT}|$ (Paths flagged that are practically unexploitable, which VACIR minimizes to 0).
- False Negatives $FN = |P_{GT} \setminus P_{det}|$ (Missed attack paths).

By defining the environment through this rigorous graph-theoretic formulation, vulnerability Management can transition from heuristic observation to formal multi-objective optimization. It processes the set P_{det} to recommend a targeted edge-removal strategy (remediation) that maximally increases $\text{Cost}(\pi^*)$ for the adversary while minimizing operational disruption to legitimate AD relationship paths.

3.3. Vulnerability management model

The core of our system is the vulnerability Management model, an integrated processing pipeline made up of four primary sub modules. First, the Detection model which serves as the entry point, which consuming raw security logs and system state information. It performs initial analysis to identify suspicious activities and generates two distinct outputs: a direct report detailing the attack path verification for immediate review, and a set of structured findings for deeper processing. Subsequently, these findings are channeled to the Analysis Model. In the final stage, the Remediation and control model takes the comprehensive insights from the Analysis Module and formulates them into an ordered set of outputs, such as a mitigation plan or a sequence of defensive actions. Each model performs a distinct function, working in concert to transform raw security data into actionable, context-aware defensive strategies.

3.3.1. The detection model

The Detection Model does not merely ingest static data; it acts as a deterministic correlation engine. It programmatically links isolated JSON attributes, Access Control Entries (ACEs), and session states across multiple files to validate the mathematical attack conditions defined in Section 3.2.2. To produce the structured VACIR Attack path output, the engine executes a multi-step algorithmic correlation process.

Phase 1: Object Resolution and Graph Initialization In the raw dataset, objects are identified by globally unique Security Identifiers (SIDs) and Object Identifiers (GUIDs) rather than human-readable names. **Phase 2: Detecting Attribute-Based Attacks (Initial Access and Account Takeover)** To populate the Initial Access and Offline Password Cracking outputs, the engine evaluates node-level attributes without requiring graph traversal. It applies boolean logic checks against the Properties array of users.json and computers.json.

- Data-to-Output Mapping: In the raw dataset, the user object for A.BIRK returns true for this property. The engine captures this, triggering the AS-REP Roasting rule. It then outputs the exact row seen in the results: Source: A.BIRK Attack Type: Initial Access Target: A.BIRK@INTERTECH.LOCAL Impact: Offline Password Cracking this implies Account Takeover.
- How Kerberoasting is Detected: The engine evaluates if (Properties.hasspn == true).
- Data to Output Mapping: For the account SVC-ILUPIEN, the raw dataset explicitly shows "hasspn": true and lists SPNs in the serviceprincipalnames array (example, VACIR_SQL/svc-ilupien). The engine identifies this as crackable service ticket material, appending it to the attack path matrix.

Phase 3: Detecting Privilege escalation through ACL Correction

The engine generates the Privilege Escalation rows in the output by performing a bipartite graph matching operation using the Aces (Access Control Entries) array found in groups.json, domains.json, and ous.json.

- The Mechanism: To determine if User A can escalate to Group B, the engine parses Group B's JSON object and loops through its Aces list. It checks the PrincipalSID (the actor) against the RightName (the permission). If the RightName matches a predefined list of exploitable primitives (GenericAll, GenericWrite, WriteDacl, WriteOwner, ForceChangePassword), the engine validates the path.
- Data-to-Output Mapping (J.PRAISE example): In the raw users.json data for J.PRAISE, the engine parses the Aces array. It identifies PrincipalType: Group and RightName: GenericWrite assigned to INTERTECH.LOCAL-S-1-5-32-544 (the Administrators SID) and the SID ending in 512 (Domain Admins). Conversely, it reads groups.json for the DOMAIN ADMINS target. When it detects that a standard user or an intermediate IT admin holds a GenericWrite or WriteOwner right over the Domain Admins object, it algorithmically generates the output row: Source: J.PRAISE links it with Attack Type: Privilege Escalation and link it to the Target: DOMAIN ADMINS@INTERTECH.LOCAL.
- Data-to-Output Mapping (Service Account Escalation): For accounts like SVC-CRINALDI and SVC-ILUPIEN, the engine detects their membership or specific ACL permissions granting them modification rights over high value groups. It outputs SVC-ILUPIEN Privilege Escalation DOMAIN ADMINS, indicating the engine successfully traced an ACL relationship from the service account's SID directly to the Domain Admins' Object Identifier.

Phase 4: Detecting Lateral Movement

Generating the Lateral Movement output requires the most complex algorithmic correlation, as it involves three distinct data points originating from computers.json.

- The Mechanism: The engine must prove an attacker can steal credentials from a machine. It executes a tripartite condition check on each computer node:
- Check Local Admins: It reads the LocalGroups array to identify which user SIDs belong to the local ADMINISTRATORS group (example, checking Name:ADMINISTRATORS@CLIENT2 .INTERTECH.LOCAL to extract the SID-544).
- Check Active Sessions: It reads the Sessions and PrivilegedSessions arrays to identify if a high-value user is currently authenticated to the machine (example, detecting UserSID-1202).
- Path Instantiation: If a compromised user SID exists in the Local Admins array, AND a target user SID exists in the Sessions array, the engine concludes the compromised user can dump the target's credentials via LSASS memory.
- Data-to-Output Mapping (CLIENT2 example): In the computers.json snippet for CLIENT2.INTERTECH.LOCAL, the engine parses the PrivilegedSessions array and discovers active user SIDs connected to ComputerSID-1307. Simultaneously, it reads the LocalGroups array and sees that Domain Admins with SID-512 are nested inside the local Administrators group. Because an attacker controlling CLIENT2 can harvest credentials from these active sessions, the engine generates the output: Source: CLIENT2.INTERTECH.LOCAL, Attack Type: Lateral Movement, Target: CLIENT2.INTERTECH.LOCAL, Impact: Attacker can steal credentials from LSASS or pivot.

Phase 5: Detecting Advanced Pathways Delegation and Public Key Infrastructure.(PKI) The engine also executes correlation logic on secondary infrastructure files to map advanced evasion and escalation paths:

- Unconstrained Delegation: When parsing computers.json for CLIENT2, the engine reads the explicit flag "unconstraineddelegation": true. The engine's logic dictates that any machine with this flag will cache the Ticket Grant ing Ticket (TGT) of any user who connects to it. This triggers a critical domain compromise alert, validating CLIENT2 as a high-priority remediation target.
- Certificate Services (AD CS) Abuse: By reading certtemplates.json, the engine evaluates the Properties block for configurations like "enrolleesuppliessubject": false combined with Aces that grant Enroll rights to unprivileged users. This allows the engine to map entirely non-credential-based privilege escalation paths, outputting alerts for offline cracking or domain admin impersonation via forged certificates.

In conclusion, the Detection Model transforms raw SharpHound JSON data into verified attack paths by correlating account attributes, ACLs, group membership, and session data. For example, A.BIRK → Initial Access/PrivEsc → A.BIRK@INTERTECH.LOCAL is detected through AS-REP Roasting, enabling offline password cracking and account takeover. Similarly, CLIENT2.INTERTECH.LOCAL → Lateral Movement → CLIENT2.INTERTECH.LOCAL is detected from active RDP/SMB sessions, allowing credential theft from LSASS. The model also identifies critical escalation paths such as SVC-ILUPIEN → Privilege Escalation → DOMAIN ADMINS@INTERTECH.LOCAL, indicating potential total domain compromise. These verified paths form the input to the Analysis Model for risk prioritization and optimized remediation.

3.3.2. The analysis model

The Analysis Model embeds the detected paths into the Security State Graph defined in Section 3.2. A detailed architecture of this modole is illustrated in Fig. 3. In this graph, AD objects such as users, computers, groups, organizational units, GPOs, certificate templates, and domain objects are represented as nodes. Relationships such as MemberOf, AdminTo, HasSession, GenericWrite, WriteDacl, WriteOwner, GetChanges, GetChangesAll,OfflineCrack, and PrivEsc are represented as weighted directed edges. Formally, VACIR analyzes the graph,

$$G = (V, E, A, W), \tag{5}$$

where V is the set of AD objects, E is the set of original and derived attack edges, A is the node attribute function, and W assigns traversal cost to each edge. Under this representation, a row such as A.BIRK → Initial Access/PrivEsc → A.BIRK@INTERTECH.LOCAL is modeled as an OfflineCrack edge from the attacker node to the A.BIRK account. Similarly, SVC-ILUPIEN → Privilege Escalation → DOMAIN ADMINS@INTERTECH.LOCAL is modeled as a privilege escalation edge from the service account to the high-value DOMAIN ADMINS group.

Once the detected paths are represented as graph edges, the Analysis Model enriches them using evidence from the raw JSON dataset. This step is important because it allows VACIR to justify each attack edge using the underlying AD configuration from which it was derived. For example, the raw users.json entry for SVC-ILUPIEN@INTERTECH.LOCAL contains "hasspn": true and service principal names such as VACIR-SQL/svc-ilupien, VACIR-SVC/svc-ilupien, and CustomSvc-svc ilupien/CLIENT1.INTERTECH.local.

The Detection Model maps these attributes to a Kerberoasting finding, while the Analysis Model interprets the resulting edge as a low-cost credential compromise path because an attacker can request a Kerberos service ticket and attempt offline cracking. This explains why the output report contains the path SVC-ILUPIEN → Privilege Escalation → SVC-ILUPIEN@INTERTECH.LOCAL, with the method Kerberoasting and the impact Offline Service Account Cracking → Account Takeover.

This same account also appears in the report as SVC-ILUPIEN → Privilege Escalation → DOMAIN ADMINS@INTERTECH.LOCAL, with the impact Total Domain Compromise. The Analysis Model therefore does not view the Kerberoasting and Domain Admin findings as unrelated alerts. Instead, it recognizes SVC-ILUPIEN as a compound-risk node because compromising the account through Kerberoasting may also expose a direct path to domain-level privilege through its privileged group relationship. This type of correlation is important because it allows VACIR to identify cases where one compromised account can serve as a bridge between initial access, account takeover, and full domain compromise.

The same graph-based correlation is also applied to lateral movement paths. In the raw computers.json data, the object CLIENT2.INTERTECH.LOCAL contains "unconstraineddelegation": true, a non-empty PrivilegedSessions array, and a LocalGroups entry for ADMINISTRATORS@CLIENT2.INTERTECH .LOCAL. The same computer object also includes privileged session records containing user SIDs and the computer SID for CLIENT2. Based on this evidence, the Detection Model generates the output row CLIENT2.INTERTECH.LOCAL → Lateral Movement → CLIENT2.INTERTECH.LOCAL, with the method Active Session (RDP/SMB). The Analysis Model then assigns higher priority to this path because it indicates that an attacker with administrative control over the host may dump LSASS memory, steal credentials from active sessions, or abuse unconstrained delegation to capture reusable Kerberos tickets. Thus, CLIENT2.INTERTECH.LOCAL is not merely treated as a computer node in the graph; it becomes a lateral movement pivot point that connects host compromise to credential theft and possible domain escalation.

After analyzing lateral movement, the model applies the same reasoning to privilege escalation paths. These paths are identified by correlating group membership and ACL evidence from groups.json, ous.json,

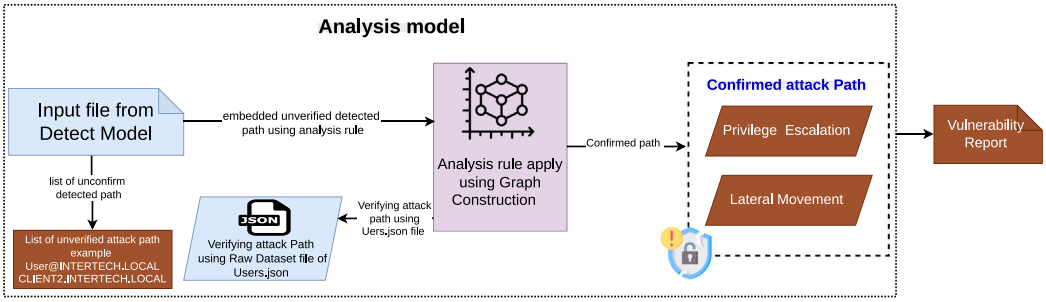


Fig. 3. The figure illustrates the end-to-end processing workflow of the VACIR Analysis Model, which operates on the structured output produced by the Detection Model to confirm, classify, and report exploitable attack paths within the AD environment.

gpos.json, containers.json, and domains.json. The output report includes examples such as C.SUDA → Privilege Escalation → SERVER OPERATORS@INTERTECH.LOCAL, with the impact Direct Login to Domain Controller, and J.PRAISE → Privilege Escalation → DOMAIN ADMINS@INTERTECH.LOCAL, with the impact Total Domain Compromise. These paths are derived from high-value group membership relationships or dangerous Access Control Entries, including GenericWrite, GenericAll, WriteDacl, WriteOwner, and AllExtendedRights. For example, the raw gpos.json, ous.json, and groups.json snippets contain ACE entries where principals are granted modification rights over sensitive objects. When such rights apply to a high-value group, policy object, or domain object, the Analysis Model treats the relationship as a privilege escalation edge because an attacker may be able to modify group membership, change permissions, take ownership, or push malicious policy changes.

Having transformed all detected findings into graph edges, the Analysis Model assigns two complementary costs to each edge: attacker traversal cost and remediation cost. The attacker traversal cost measures how easy or difficult it is for an adversary to exploit a given path. For instance, AS-REP Roasting paths such as A.BIRK → A.BIRK@INTERTECH.LOCAL receive a low traversal cost because the attack can be performed offline once Kerberos material is obtained. Kerberoasting paths such as SVC-ILUPIEN → SVC-ILUPIEN@INTERTECH.LOCAL are also treated as relatively low-cost because the service ticket can be requested and cracked offline. Lateral movement paths such as CLIENT2.INTERTECH.LOCAL → CLIENT2.INTERTECH.LOCAL receive low traversal cost when active privileged sessions are present, since credentials may be harvested from LSASS. In contrast, remediation cost measures the operational impact of removing or modifying the vulnerable edge. Removing an unnecessary group membership may have relatively low remediation cost, whereas disabling a production service account or changing Kerberos encryption settings may require additional validation to avoid service disruption.

Using these two cost dimensions, VACIR formulates remediation as a multi-objective optimization problem. Each potential defensive action is represented as a Boolean decision variable $x_e \in \{0, 1\}$, where $x_e = 1$ means that edge e is selected for remediation and $x_e = 0$ means that the edge remains unchanged. The objective is to minimize the total remediation cost while ensuring that every high-risk path to a critical target is disrupted. In practical terms, if several paths converge on DOMAIN ADMINS@INTERTECH.LOCAL, such as A.GEISEL → DOMAIN ADMINS@INTERTECH.LOCAL, J.PRAISE → DOMAIN ADMINS@INTERTECH.LOCAL, SVC-ILUPIEN → DOMAIN ADMINS@INTERTECH.LOCAL, and Z.VARIAN → DOMAIN ADMINS@INTERTECH.LOCAL, the Analysis Model does not recommend isolated fixes for each alert. Instead, it identifies shared structural causes and selects the smallest set of remediation actions that breaks the dangerous paths while preserving legitimate administrative access.

As shown in Algorithm 2, analysis begins by constructing lookup tables to map object IDs to principal names (line 3). The procedure then systematically identifies attack paths based on common adversarial techniques. Step 1 (lines 5–11) analyzes lateral movement.

Opportunities by identifying active user sessions on compromised computers, a technique commonly known as session hijacking. Step 2 (lines 13–19) discovers privilege escalation vectors by examining the memberships of high-value target groups. Step 3 (lines 23–30) identifies protocol-level vulnerabilities related to Kerberos abuse, specifically flagging users susceptible to AS-REP Roasting (DontRequirePreAuth) and Kerberoasting (HasSPN). Finally, Step 4 (lines 32–34) involves

Algorithm 2 VACIR Attack Path Discovery and Analysis

Require: Extracted JSON Datasets $D_{json} = \{Users, Groups, Computers\}$
Ensure: Deduplicated Attack Path Report R_{paths}
1: **procedure** VacirEngineAnalysis(D_{json})
2: $P \leftarrow \emptyset$ //Initialize empty set for attack paths
3: $L_{map} \leftarrow$ BuildLookupTables(D_{json}) //Map ObjectIDs to Principal Names
4: **Step 1: Analyze Lateral Movement (Session Hijacking)**
5: **for all** $comp \in D_{json}.Computers$ **do**
6: **if** HasActiveSessions($comp$) **then**
7: **for all** $sess \in comp.Sessions$ **do**
8: $user \leftarrow$ ResolveName($sess, L_{map}$)
9: $P \leftarrow P \cup$ CreatePath($user, comp,$ "Lateral Movement")
10: **end for**
11: **end if**
12: **end for**
13: **Step 2: Analyze Privilege Escalation (Group Memberships)**
14: **for all** $group \in D_{json}.Groups$ **do**
15: **if** IsHighValueTarget($group$) **then**
16: **for all** $member \in group.Members$ **do**
17: $user \leftarrow$ ResolveName($member, L_{map}$)
18: $P \leftarrow P \cup$ CreatePath($user, group,$ "Privilege Escalation")
19: **end for**
20: **end if**
21: **end for**
22: **Step 3: Analyze Protocol Vulnerabilities (Kerberos Abuse)**
23: **for all** $user \in D_{json}.Users$ **do**
24: **if** $user.DontRequirePreAuth ==$ True **then**
25: $P \leftarrow P \cup$ CreatePath($user, user,$ "AS-REP Roasting")
26: **end if**
27: **if** HasSPN($user$) **then**
28: $P \leftarrow P \cup$ CreatePath($user, user,$ "Kerberoasting")
29: **end if**
30: **end for**
31: **Step 4: Post-Processing**
32: $P_{unique} \leftarrow$ DeduplicatePaths(P)
33: $R_{paths} \leftarrow$ ExportToDataFrame(P_{unique})
34: **return** R_{paths}
35: **end procedure**

post-processing the discovered paths to remove duplicates and export the results into a structured format. The resulting set of unique attack paths, R-paths, forms the foundational input for the multi-objective optimization solver. By evaluating the entire attack surface holistically, the engine moves beyond heuristic-based approaches to produce a mathematically optimized and context-aware security plan tailored to the specific AD environment.

Overall, the Analysis Model transforms the verified findings from the Detection Model into a weighted, prioritized, and optimizable attack graph. By combining raw JSON evidence such as hasspn, dontreqpreauth, unconstraineddelegation, Aces, Local Groups, Privileged Sessions, and domain replication permissions with concrete paths from the attack-path report, the model enables VACIR to move beyond attack-path discovery. It provides the analytical foundation required to compute minimal, high-impact, and operationally safe remediation strategies against lateral movement and privilege escalation in AD.

3.3.3. Integer linear programming (ILP) formulation

Let $\mathcal{P}$ represent the set of all detected viable attack paths from the entry points to the high-value target set $\mathcal{T}$. Let $\mathcal{A}$ be the universe of candidate remediation actions (e.g., revoking a specific ACL, enforcing AES Kerberos encryption, or stripping a group membership). Each action $a \in \mathcal{A}$ is assigned a binary decision variable $x_a \in \{0,1\}$, where $x_a = 1$ denotes the execution of the action. Let $c_a > 0$ denote the operational cost associated with action a. The primary objective is to formulate a minimum-cost hitting set that severs all adversarial pathways. We minimize the total remediation cost:

$$\min \sum_{a \in \mathcal{A}} c_a x_a \tag{6}$$

Subject to path-blocking constraints, ensuring every path $p \in \mathcal{P}$ is broken by at least one action in its associated action subset $\mathcal{A}(p)$:

$$\sum_{a \in \mathcal{A}(p)} x_a \geq 1 \quad \forall p \in \mathcal{P} \tag{7}$$

To prevent service disruption, we introduce strict operational-safety constraints C. For any operationally critical entity e (such as the built-in ADMINISTRATOR), we enforce a hard non-interference constraint:

$$x_{a,e} = 0 \quad \forall e \in C \tag{8}$$

subject to the constraints above. The resulting plan is a minimum cost hitting set over attack paths, with explicit operational safety restrictions.

Algorithm 3 VACIR ILP Remediation Optimizer

Require: Attack paths $\mathcal{P}$, Candidate actions $\mathcal{A}$, Safety constraints C
Ensure: Minimum-cost intervention plan R_{opt}

procedure ILP_Optimizer($\mathcal{P}, \mathcal{A}, C$)
  Initialize problem model O as MINIMIZE
  for each $a \in \mathcal{A}$ **do**
    Define variable $x_a \in \{0,1\}$ with cost c_a
  end for
  Set $O_{objective} \leftarrow \sum(c_a \cdot x_a)$
  for each $p \in \mathcal{P}$ **do**
    $\mathcal{A}(p) \leftarrow$ Extract edges for path p
    Add constraint: $\sum_{a \in \mathcal{A}(p)} x_a \geq 1$
  end for
  for each entity $e \in C$ **do**
    Add constraint: $x_{a,e} = 0$
  end for
  $R_{opt} \leftarrow$ Solve O using CBC/PuLP
  return R_{opt}
end procedure

3.3.4. Remediation and control model

The Remediation and Control model bridges the gap between the theoretical analysis performed by the engine and practical, real-world implementation. It serves as the framework's "actuator", responsible for both executing the computed remediation strategies and managing the experimental workflow to ensure scientifically valid and repeatable validation. This component transforms the abstract set of defensive actions into concrete changes within the lab environment and automates the entire validation lifecycle. The orchestrator's responsibilities are twofold:

Remediation Plan Execution: VACIR considers five categories of defensive interventions:

- **ACL Hardening:** (Removal of excessive GenericWrite, WriteDacl, or WriteOwner permissions.) This action directly prevents Privilege Escalation through Object Takeover. By revoking misconfigured AD Security Descriptors, it prevents adversaries from manipulating domain objects. For example, as demonstrated in our case study, removing GenericWrite prevents a compromised standard user (e.g., a.birk) from maliciously adding themselves to high-privilege groups (example, CityC IT Admins) or granting themselves DCSync rights to initiate a domain takeover.

- **Group Membership Correction:** (Enforcing least-privilege group assignments). This mitigates both Lateral Movement and direct Privilege Escalation. Over-permissioned accounts and deep group nesting create hidden, high-value attack paths. By systematically stripping unnecessary administrative memberships, this action shrinks the attack surface, limits the blast radius of a compromised account, and severs the direct graph edges that adversaries exploit to bypass network compartmentalization.

- **Kerberos Hardening:** (Disabling Rivest Cipher 4 (RC4), Data Encryption Standard(DES) and enforcing Advanced Encryption Standard(AES)) This defense neutralizes Offline Credential Cracking, specifically targeting Kerberoasting and AS-REP Roasting vulnerabilities. By ensuring that all user accounts require Pre-Authentication (DontRequirePreAuth = False) and enforcing computationally expensive AES encryption for Service Principal Names (SPNs), attackers are prevented from easily extracting and cracking service ticket hashes offline.

- **Delegation Restriction:** (Removing unconstrained delegation configurations.) This blocks advanced Token Theft and Credential Relaying. Unconstrained delegation is a critical misconfiguration that allows a compromised host to cache the Ticket Grant ing Tickets (TGTs) of any user who authenticates to it. Revoking this setting ensures that adversaries cannot harvest high-privilege tickets (such as a Domain Admin's TGT) from a compromised server to pivot laterally across the domain.

- **Session Hygiene Controls:** Limiting credential exposure via privileged session isolation. The ILP/SMT solver selects the minimal subset of actions that eliminates all high-risk attack paths while satisfying operational constraints.

The primary function is to take the remediation plan generated by the Analysis Engine, a list of specific actions such as removing a user from a group or revoking a risky permission—and translate it into executable commands. The orchestrator automatically generates and deploys PowerShell scripts that apply these changes directly to the Domain Controller. This automated execution guarantees that the remediation is applied accurately and consistently, eliminating the potential for manual error and ensuring the integrity of the experiment.

State Management and Experimental Control: To enable fair, comparative analysis, the orchestrator manages the state of the entire virtualized environment using Infrastructure-as-Code (IaC) principles

Algorithm 4 VACIR Remediation Algorithm

Require: D_{json}, Methods: *Baseline*, *VACIR-Opt*
Ensure: R_{comp}: Performance Comparison Report
procedure OrchestrateValidation(D_{json})
  $P \leftarrow$ VacirAnalysis(D_{json}) //Call Alg. 2
  $TTC_{base} \leftarrow$ ValidateCompromise(P) //Initial Time-to-Compromise

Step 1: Baseline Remediation
  $Plan_b \leftarrow$ GeneratePlan(Base, P)
  RevertSnapshot (Post-Recon)
  ApplyRemediation($Plan_b$) //Run PowerShell scripts
  $TTC_b \leftarrow$ ValidateCompromise(P)

Step 2: VACIR Remediation
  $Plan_v \leftarrow$ OptimalPlan(P, ILP/SMT)
  RevertSnapshot (Post-Recon)
  ApplyRemediation($Plan_v$) //Run PowerShell scripts
  $TTC_v \leftarrow$ ValidateCompromise(P)

Step 3: Generate Report
  $R_{comp} \leftarrow$ GenReport(TTC_{base}, TTC_b, TTC_v)
  return R_{comp}
end procedure

and hypervisor-level snapshots. It automates the process of reverting the lab to a specific baseline (e.g., the "post-reconnaissance" state) before applying a remediation plan. This ensures that different remediation strategies (e.g., a baseline plan vs. the VACIR-optimized plan) are tested under identical conditions, eliminating configuration drift and experimental bias.

The complete workflow managed by the orchestrator, from initial analysis to final validation, is formalized in Algorithm 3.

4. Evaluation

4.1. Experimental environment

All empirical evaluations were conducted within a fully isolated and controlled cyber range on a dedicated research workstation equipped with an Intel(R) Core (TM) i5-13500 CPU (20-core 2.50 GHz) and 16 GB DDR4 RAM. The environment was virtualized using Oracle VirtualBox version 7.0, with all virtual machines interconnected via an isolated "Host-Only" network to prevent external interference and ensure safe execution of adversary emulation tasks. The architecture of this cyber range, alongside the specified hardware constraints, is foundational to the reproducibility of our execution time metrics.

- **Core System Components**

The cyber range is comprised of the following key virtual machines as shown in Fig. 4 below.

Domain Controller (DC): A single virtual machine running Windows Server 2025 (Hostname: WINSERVER, IP: 192.168.56.113). This server hosts the INTERTECH.local AD domain, serving as the central authority for identity, authentication, and policy.

This hierarchical OU design, with its graduated privilege boundaries, enables realistic simulation of lateral movement and privilege escalation scenarios. Attacker Machine: A virtual machine running Kali Linux (IP: 192.168.56.114). This machine is designated for adversary emulation and is equipped with a standard suite of red-team tools used for reconnaissance, exploitation, and post-exploitation, including Nmap, Responder, Hashcat, Evil-WinRM, and SharpHound.

Logical Domain Architecture and Threat Objectives

The `INTERTECH.LOCAL` domain is geographically segmented into Cities A through D. Each city is further divided into departmental OUs (e.g., HR, Finance, and Management) containing standard user accounts, alongside a dedicated **OU: IT Department** housing privileged accounts responsible for localized infrastructure administration. As illustrated in Fig. 5 of Section 4.2, this two-tier segregation (geographical and functional) mirrors common enterprise practices where IT operations are decentralized across regional offices, yet standardized departmental structures are maintained uniformly. The visual representation utilizes hierarchical connecting lines to denote parent–child relationships, dotted boxes to indicate network-accessible shared resource areas, and distinct AD user icons to separate standard users from administrative accounts. While this tiered administrative model is theoretically designed to enforce least-privilege principles, ACL misconfigurations and excessive group nesting inherently create unintended privilege escalation pathways across organizational boundaries. Notably, the presence of sensitive credentials on a shared file system accessible from the Domain Controller represents a high-risk attack primitive that VACIR successfully identifies as part of its lateral movement detection capabilities (contributing to the 12 lateral movement paths reported in Table 3 which can be seen in Section 4.2.

From an adversarial perspective, the objective is to obtain high-privilege access such as effective control of the Domain Administrator account or the Domain Controller through a sequence of lateral movement and privilege escalation operations. The adversary is assumed to be capable of conducting domain reconnaissance, enumerating security relationships, and abusing configuration weaknesses (e.g., excessive ACL permissions and delegation flaws). The threat model focuses strictly on attack paths that emerge from known vulnerabilities, documented AD misconfigurations, and observable security relationships rather than undisclosed zero-day exploits. Conversely, from the defender's perspective, the laboratory environment is configured using default AD deployment practices to emulate a realistic enterprise setting. We assume a standard administration model in which the defender has basic domain management knowledge but lacks comprehensive visibility into the chained security implications of complex AD relationships.

This structured hierarchy provides the foundational dataset for evaluating VACIR's ability to map attack paths spanning multiple organizational units, detect cross-boundary privilege escalation, and recommend minimal-disruption remediation strategies. As detailed in Fig. 7 of Section 4.4.1, critical vulnerability indicators including 33 Kerberoastable users, 27 AS-REP roastable users, and 2 unconstrained delegation computers expose the domain to severe credential theft and lateral movement risks. Furthermore, the presence of 77 domain root ACL entries and 896 OU-level ACL entries reflects a highly complex and potentially misconfigured permission structure. Overlapping high-privilege memberships, such as 51 Domain Admins and 30 Remote Desktop Users, significantly amplify the structural risk of privilege escalation.

- **Automated Provisioning and Vulnerability Injection**

Controlled vulnerabilities were injected deterministically, 32 users configured with weak Kerberos encryption (RC4-HMAC-MD5), 12 service accounts with extractable SPNs and passwords in rockyou.txt, 8 ACL misconfigurations granting GenericWrite on high-value groups, 4 computers with unconstrained delegation enabled and 12 simulated admin sessions on workstations (RDP/SMB). This is to ensure consistency and eliminate configuration drift; the environment was provisioned using an Infrastructure-as-Code (IaC) approach. Custom PowerShell orchestration scripts were executed on the Domain Controller to programmatically:

Create a structured identity hierarchy with specific Organizational Units (OUs) such as DomainAdmins, ServiceUsers and StandardUsers. Provision twenty-five user accounts with predefined roles (Domain

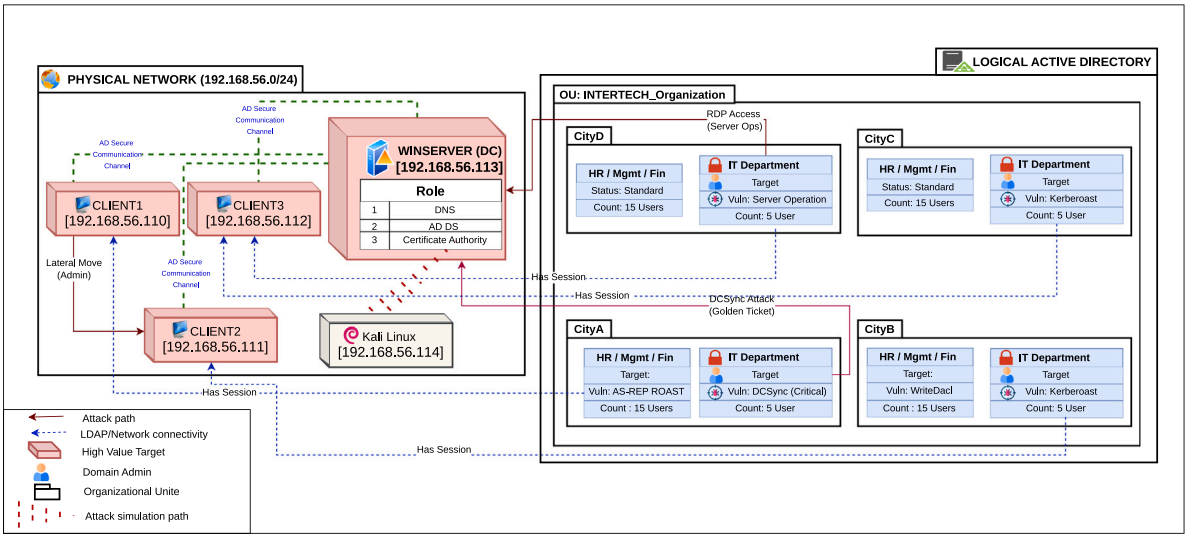


Fig. 4. Show an intersection of physical and logical attack surfaces. The left panel shows the physical network (192.168.56.0/24 subnet) with three domain-joined clients and the WINSERVER domain controller. The right panel displays the logical AD hierarchy: INTERTECH.local root domain → organizational units (CityA, CityB, CityC) → departmental Ous → user/computer objects. Injected vulnerabilities include: weak NTLM hashes susceptible to offline cracking, overprivileged ACLs granting GenericWrite to CityC IT Admins, service accounts with Kerberoastable SPNs, and active admin sessions on workstations enabling credential theft.

Admin, Service Account, Standard User). Deterministically inject a set of controlled vulnerabilities aligned with Lateral movement and privilege escalation including Kerberoastable Service Principal Names (SPNs) and Unconstrained Delegation settings.

- **State Management and Repeatability**

The integrity and repeatability of experiments are guaranteed through the use of VirtualBox snapshots. After the automated provisioning was complete, a baseline "Pre-experimental Clean State" snapshot of the entire cyber range was captured. This snapshot allows the environment to be reset to an identical, pristine state before each experimental run, ensuring that all tests are conducted under uniform conditions and that results are directly comparable.

4.2. Experimental setup

The empirical validation of the VACIR framework is contingent upon a rigorous and reproducible data acquisition methodology. To this end, we designed and executed a systematic experimental protocol that operationalizes the composite attack function, $F = E.X.A.K.C.R$, previously defined in Section 3.2. The primary objective of this protocol is to generate a high-fidelity dataset that authentically represents the post-compromise attack surface from an adversary's perspective. This process methodically transitions the testbed environment from an initial zero-knowledge state (S_0) to a state of full domain visibility, culminating in the construction of the fully instantiated Security State Graph, G, which serves as the substrate for our subsequent optimization analyses. The protocol is structured as shown in Fig. 5.

Phase 1: External Reconnaissance and Network Topology Mapping (R):

This initial phase models an external adversary with no a-priori knowledge of the target infrastructure. The execution of the reconnaissance function, $R(S_0)$, involves a systematic interrogation of the network layer. We employed Nmap to perform service discovery and version detection across the isolated 192.168.56.0/24 subnet. The function's output is a topology mapping that identifies live hosts and their exposed service ports. The successful identification of the Domain Controller (192.168.56.113) was achieved by fingerprinting its characteristic service signatures, including Kerberos (TCP/88), LDAP (TCP/389), and SMB (TCP/445), thus confirming the presence of an AD environment and establishing a primary target for the next phase.

Phase 2: Credential Compromise and Foothold Establishment (A.K.C):

This phase simulates the acquisition of an initial foothold by composing three sub-functions to achieve authenticated access.

- **Credential Hash Interception (C):** The first step leverages inherent weaknesses in legacy name resolution protocols common to Windows environments. Responder was utilized to perform LLMNR/NBT-NS poisoning, compelling domain clients to surrender NTLMv2 challenge-response hashes. This function transforms the state from pure reconnaissance to one where raw credential material has been obtained.
- **Offline Credential Cracking (K):** The intercepted NTLMv2 hashes were subjected to a high-throughput, offline dictionary attack using Hashcat. This computational function solves the inverse problem of recovering the plaintext password from its corresponding hash, detailed analysis in Fig. 6 of Section 4.4.1

- **Authenticated Session Establishment (A):** The recovered credential pair serves as the input to the access function. Using Evil-WinRM, an authenticated, interactive remote shell was established on a domain-joined host. This step marks a critical state transition, moving the adversary from an external, unauthenticated position to an internal, authenticated one, thereby providing a platform for privileged enumeration.

Phase 3: Privileged Enumeration and Graph Data Extraction (E):

Once inside the network perimeter with a valid user context, the enumeration function, E, was executed. This phase employs the SharpHound collector, a tool specifically designed to perform recursive, graph-oriented enumeration of AD. The collector was run from the compromised host to systematically query the domain for all security principals (V_U, V_C, V_G), organizational structures, and the explicit and implicit relationships (E) that connect them. This includes ACLs, group memberships, session information, and delegation settings. The output of this function is a comprehensive, structured data archive containing a series of JSON files (Fig. 8 of Section 4.4.1), which represents the totality of the adversary's knowledge post-compromise.

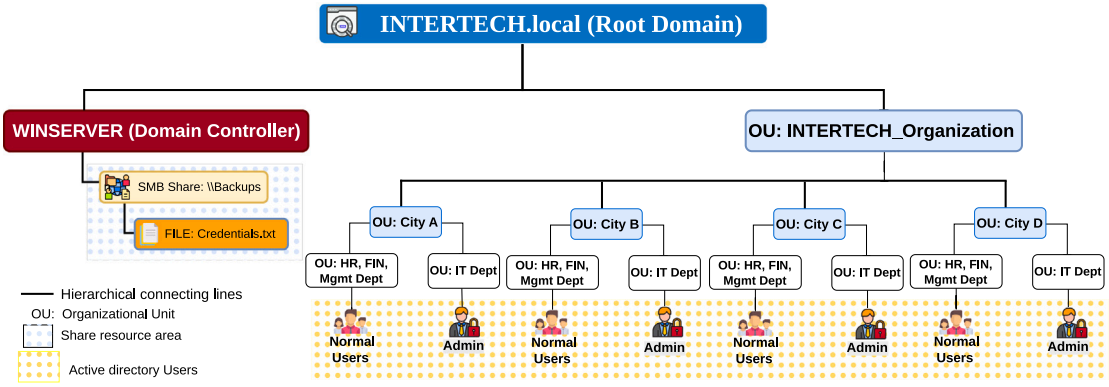


Fig. 5. Logical Hierarchy of the INTERTECH.local AD Domain.

Phase 4: Semantic Transformation and Graph Instantiation. The final phase of the protocol bridges the raw data artifact with our formal model. The collected JSON files undergo a semantic transformation process within the VACIR pipeline. This involves ingestion, normalization, and the mapping of raw data objects and relationships to their corresponding formal entities in the Security State Graph, G=(V,E). For instance, a "MemberOf" relationship in the data is instantiated as a directed edge in G. This critical step ensures that the resulting graph is not a theoretical construct but a direct, empirical representation of the attack paths and structural vulnerabilities discoverable by an adversary. By adhering to this structured, adversary-centric protocol, we guarantee both the ecological validity and reproducibility of our experimental foundation, creating a robust dataset for the rigorous evaluation of VACIR's detection and optimization capabilities under realistic conditions.

4.3. Evaluation metrics

To avoid circularity in evaluation, ground truth (GT) for each dataset was derived from independent, publicly documented attack paths. For the SEVENKINGDOMS.LOCAL dataset (m4lwhere, 2024b) and NORTH.SEVENKINGDOMS.LOCAL datasets (m4lwhere, 2024a), we utilized the official documentation from the Orange Cyberdefense GOAD (Game of Active Directory) v2.0 lab. For PHANTOM.CORP, dataset (SpecterOps, 2025b) ground truth was established using the BloodHound Community Edition version v9.6.0, with its Sample Dataset documentation provided by SpecterOps Overall, the results validate the effectiveness of VACIR. The proposed framework achieves a balance between accuracy, scalability, and computational efficiency.

4.4. Comparing our method with other published work and running our engine on publicly available dataset

4.4.1. RQ1: Comparing our method with other methods

To answer RQ2, we benchmark VACIR against representative graph-based AD analysis baselines on multiple publicly available datasets (VACIR, BloodHound (SpecterOps, 2025b), Sevenkingdoms (m4lwhere, 2024b), and North (m4lwhere, 2024a)). The comparison uses the same ground-truth vulnerability labels (GT) per dataset and reports standard classification measures (TP, FP, FN) together with Precision, Recall, F1-score and execution time. Two baseline methods are considered: BloodHound Analyzer, which reflects conventional BloodHound-centric analysis, and CoZure, included for completeness; however, CoZure did not produce detectable outputs on the evaluated datasets (reported as undetected), and therefore cannot be meaningfully assessed beyond the shared confusion-matrix values reported in Table 2.

- **Detection Correctness**

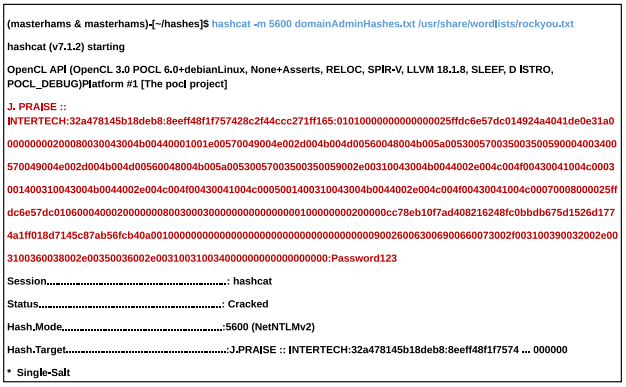


Fig. 6. Hashcat execution against domainAdminHashes.txt using hash mode 5600 (NetNTLMv2) and the rockyou.txt wordlist. The attack resulted in a successful crack of the target user J.PRAISE's hash, exposing the password "Password123".

Across all four datasets, VACIR achieves Precision = 1.00, indicating that every reported finding corresponds to a genuinely exploitable path. This outcome is not incidental it is the direct consequence of two design decisions in the methodology.

Firstly, the VACIR Detection Model (Section 3.3.1) implements a deterministic, rule-based pattern matching process that is applied exhaustively across the complete vertex set V_U of the Security State Graph G. On the INTERTECH.local domain, this traversal was applied over 425 domain users, identifying 27 accounts vulnerable to AS-REP Roasting (including A.BIRK, A.SCLAFANI, C.ANSARI, D.PLUMB, E.SANDOR, H.GROS, I.AUTHEMENT1, J.BEVERLIN, J.CASTANEDA1, O.MCRAY, R.ADKINS, S.BATCHELOR, and W.BUMGARNER, among others), 4 lateral movement paths targeting CLIENT1–CLIENT3 and WINSERVER via Active Session (RDP/SMB), and 128 privilege escalation paths spanning Group Membership abuse. The engine detected 186 users and 129 total vulnerabilities (TP = 129, FP = 0) in 0.29 s. By contrast, BloodHound Analyzer which relies on query-driven, analyst-directed enumeration detected only 59 users and 59 vulnerabilities while producing 70 false positives, requiring 88.33 s of run time. This two-order-of-magnitude difference in both coverage and speed (129 TP vs. 59 TP; 0.29s vs. 88.33s) directly reflects the absence of automated exhaustive traversal in the baseline.

Second, output reveals a structurally significant pattern: the 38 SVC-prefixed service accounts (SVC-ABIRK, SVC-AGOLTZ, SVC-CANSARI, SVC-CIBARRA, SVC-CRINALDI, SVC-CSHADDIX, SVC-DBANDA, SVC-DBIONDO, SVC-FSTERLING, SVC-HSKIPPER, SVC-ILUPIEN, SVC-ISELVIDGE, SVC-JDONATI, SVC-KCHICO, SVC-LIRVIN, SVC-LMCCUSKER, SVC-LPAZ, SVC-MGARTIN, SVC-NBRISKI,

Table 2
VACIR comparison with other methods.

Dataset	Method	Users Det.	No. of Host Det.	Vuln Det.	TP	FP	FN	Precision	Recall	F1	Exec. time (seconds)
VACIR	Our method (VACIR)	186	4	129	129	0	10	1.00	0.92	0.96	0.29
	BloodHound Analyzer (SpecterOps, 2025a)	59	1	59	59	70	11	0.46	0.84	0.59	88.33
BloodHound (SpecterOps, 2025b)	Our method (VACIR)	53	21	37	37	0	16	1.00	0.69	0.82	0.20
	BloodHound Analyzer (SpecterOps, 2025a)	31	10	31	31	10	69	0.75	0.31	0.44	86.32
Sevenkingdoms (m4lwhere, 2024b)	Our method (VACIR)	15	1	5	12	0	3	1.00	0.80	0.89	0.18
	BloodHound Analyzer (SpecterOps, 2025a)	5	1	5	4	4	14	0.50	0.22	0.31	74.42
North (m4lwhere, 2024a)	Our method (VACIR)	16	2	9	9	0	6	1.00	0.60	0.75	0.18
	BloodHound Analyzer (SpecterOps, 2025a)	6	1	6	6	3	10	0.67	0.38	0.49	64.32

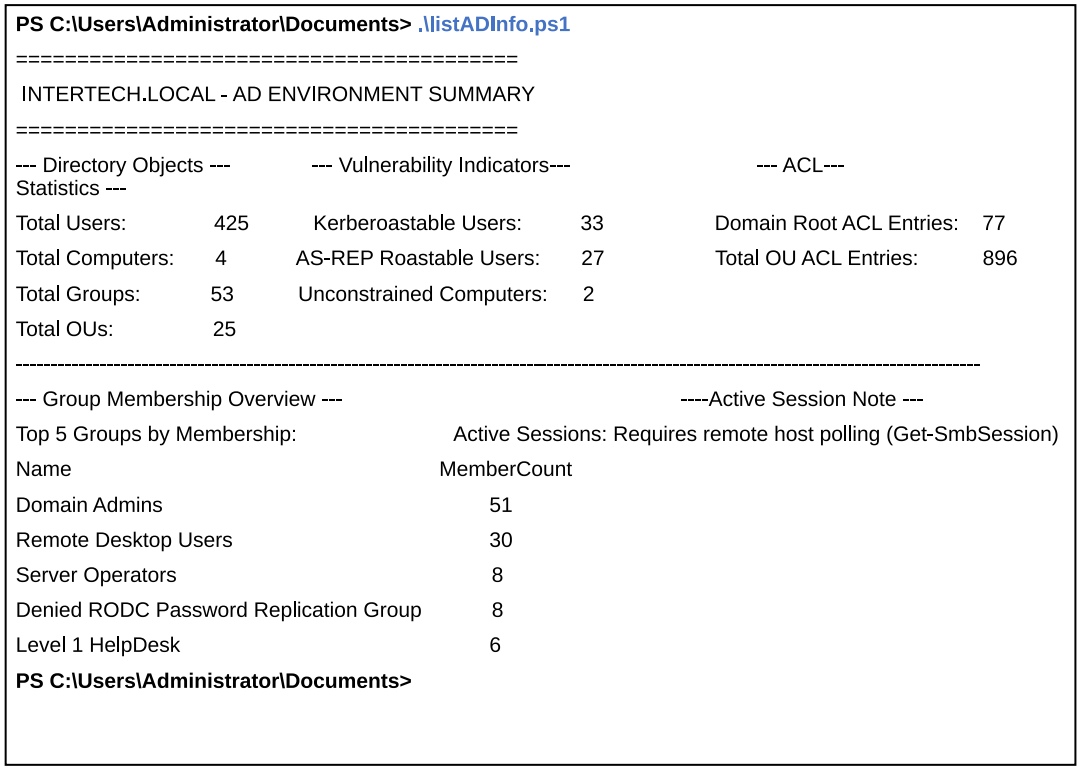


Fig. 7. Overview of the Experimental AD Dataset Including Object Counts, Vulnerability Indicators, and ACL Statistics.

SVC-SBAXLEY, SVC-TGOGGANS, SVC-TNOCERA, SVC-TSTALLER, SVC-TTAITT, and SVC-VLAFLEUR) each appear twice in the privilege escalation output once targeting DOMAIN ADMINS via Group Membership and once via Kerberoasting. This dual-vector structure means each account is simultaneously exploitable through two independent, low-cost paths. A heuristic remediation approach addressing only one vector leaves the domain fully exposed through the other. The VACIR Analysis Module (Section 3.3.2) addresses this through a formal multi-objective ILP/SMT optimization that simultaneously evaluates all detected paths,

$$P_{\text{det}} = P_{LM} \cup P_{PE} \cup P_{AT}$$

and selects the minimum-cardinality intervention set that closes all viable adversarial paths while satisfying operational constraints that prevent DoS conditions.

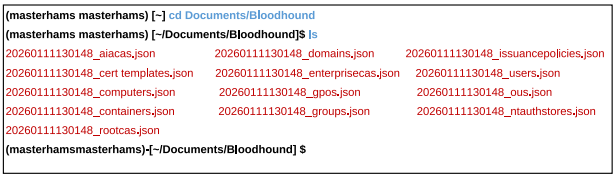


Fig. 8. Command-line view of the BloodHound ingestor output files, containing enumerated AD objects (users, groups, computers, OUs, GPOs, containers) and AD Certificate Services (AD CS) components (AIA CAs, root CAs, enterprise CAs, certificate templates, issuance policies, and NT auth stores) in JSON format for further analysis.

- **Baseline Comparison**

CoZure: was initially considered as a baseline methodology. However, CoZure is architecturally designed exclusively for Azure AD (Entra ID) environments, operating via the Microsoft Graph API. Because our evaluation datasets consist of on-premises SharpHound collections relying on LDAP, RPC, and SAMR protocols, CoZure yielded no actionable output.

BloodHound Analyzer: BloodHound Analyzer refers specifically to BloodHound Community Edition v8.7.0 The comparative analysis utilized the standard 'Shortest Paths to Domain Admin' Cypher query set. To ensure a standardized comparison for execution time, the timer boundaries for the baseline were defined as the full duration from query initiation in the Neo4j backend to the final rendering of the attack path graph. Under these identical hardware constraints, VACIR processed the graph in sub-second runtimes (≤ 0.29 seconds), whereas the query-driven baseline required 88.33 s.

4.4.2. RQ2: Running our engine on publicly available dataset

To evaluate the generalizability of the VACIR detection engine, we apply the engine on INTERTECH.local dataset and three independently sourced, publicly available AD datasets: the BloodHound Community Edition sample (PHANTOM.CORP), the SEVENKINGDOMS dataset (SEVENKINGDOMS.LOCAL), and the NORTH dataset (NORTH.SEVENKINGDOMS.LOCAL) (Detailed analysis in Table 3 of Section 4.5).

We evaluate generalizability by running VACIR on INTERTECH.local and three publicly available SharpHound datasets: PHANTOM.CORP, SEVENKINGDOMS.LOCAL, and NORTH.SEVENKINGDOMS.LOCAL. Each dataset is processed without modification.

Session parsing and lateral-movement counting. Session Parsing and Lateral Movement Derivation. In modern BloodHound Community Edition (CE) schemas, session data is encapsulated within a structured JSON object containing collection metadata (for example, Collected, FailureReason) alongside the array of actual session objects (Results). To ensure strict empirical validity and eliminate parsing artifacts such as artificially inflating lateral movement counts by iterating over metadata keys the VACIR Detection Model employs a targeted extraction algorithm. The engine explicitly isolates the Results and PrivilegedSessions arrays, discarding all surrounding metadata. A lateral movement opportunity is mathematically instantiated as an edge $e_{LM} \in E$ if and only if two conditions are met: (1) a valid session record exists for a target user u_{target} on host c, and (2) the compromised principal u_{comp} holds a verified execution primitive (for example, AdminTo, CanRDP, PSRemote) over c. To guarantee uniqueness, session edges are rigorously deduplicated using the structural tuple $(u_{comp}, c, u_{target})$. This ensures that the lateral movement metrics provided to the ILP optimizer such as the 12 verified multi hop chains in the INTERTECH.LOCAL baseline represent uniquely actionable adversarial pivot points rather than redundant enumeration artifacts.

Privilege escalation vs privileged membership inventory. VACIR distinguishes (i) *privileged principals* (e.g., accounts already in Domain Admins) from (ii) *exploitable escalation paths* (i.e., transitions that allow a lower-privilege principal to reach a higher-privilege target via an abuse primitive such as `GenericWrite`, `WriteDacl`, `WriteOwner`, delegation abuse, or replication rights). Built-in accounts such as `Administrator` being members of `Domain Admins` are reported only as *privileged membership inventory*, not as an "exploitable path". Only the latter category is used when computing TP/FP/FN in Table 2.

KRBTGT anomalies and golden ticket prerequisite modeling. Kerberoasting inherently requires the extraction of a crackable service ticket. Because the built-in krbtgt account is disabled by default and secured by a high-entropy, machine-generated key, the mere presence of a Service Principal Name (SPN) on this account does not constitute a computationally viable exploit edge. In the revised VACIR framework, the Detection Model classifies an SPN on krbtgt strictly as a configuration anomaly rather than an exploitable Kerberoasting pathway. This strict classification is vital for the downstream Integer Linear Programming (ILP) optimizer, as it prevents the solver from assigning remediation costs to mathematically unexploitable edges. Consequently, VACIR now explicitly isolates Golden Ticket attack vectors, instantiating an exploit edge if and only if the dataset contains deterministic evidence of Active Directory replication abuse primitives (e.g., an attacker possessing both GetChanges and GetChangesAll privileges over the domain root object) or equivalent preconditions for Domain Controller compromise.

Authorized inventory vs. Exploitable graph traversal. VACIR framework mathematically partitions group relationships into two distinct sets: Authorized Inventory (E_{auth}) and Exploitable Paths ($E_{exploit}$). The presence of the built-in Administrator account within the DOMAIN ADMINS group is classified strictly as E_{auth}. It represents the intended, baseline state of the directory and is explicitly excluded from the exploitable path metrics ($|\mathcal{P}|$). Conversely, an Exploitable Path ($E_{exploit}$) is defined as a graph transition allowing a lower-privileged principal to maliciously traverse to a higher privileged state via an abuse primitive (for example, GenericWrite, WriteDacl, or unconstrained delegation pivoting).By strictly mapping E_{auth} relationships to the operational safety constraints of the ILP optimizer (for example, enforcing $x_a = 0$ for the built-in Administrator), VACIR ensures that the objective function never recommends severing legitimate administrative access. Furthermore, the generation of the remediation strategy actively exercises the weighted path formalism defined in Section 3.2.3. The ILP solver performs deep graph traversal to map all converging, multi-hop chains across $E_{exploit}$, computing the global minimum-cost hitting set rather than relying on isolated, single node attribute checks.

Per-dataset findings (summary). VACIR outputs three classes of evidence-backed findings: (1) account-takeover conditions (e.g., AS-REP roastable, Kerberoastable service accounts), (2) lateral-movement opportunities (session-present + execution primitive on the host), and (3) privilege-escalation transitions (ACL/delegation/replication abuse leading to higher privilege). We report totals and performance metrics in Table 2 and a cross-dataset feature summary in Table 3 in Section 4.5 below.

4.5. RQ3 case study

To evaluate the empirical validity of our detection engine, a comprehensive case study was conducted focusing on the initial anomalous entity flagged in the system report: the user account a.birk. This phase simulates a post-compromise scenario, assuming the credentials for a.birk were exfiltrated as detailed in Section 4.2. The objective is to demonstrate a viable attack trajectory from a standard workstation user to a high-privileged City-Level Domain Administrator, leveraging specific structural misconfigurations within the AD environment.

Table 3
Comparison of VACIR-AD Across Different Dataset Environments.

Feature		Public data (BloodHound) (SpecterOps, 2025b)	Public data (Sevenkingdoms) (m4lwhere, 2024b)	Public data (NORTH) (m4lwhere, 2024a)	VACIR-AD local lab environment (INTERTECH.local)
Total Users		53 users	15 users	16 users	420 users
Users with Elevated Risk of Compromise		37 users	7 users	12 users	159 users
Identified Attack Paths		Highly targeted Attack Paths	Highly targeted Attack Paths	Highly targeted (Specific Kill Chains)	Highly Targeted Attack Paths Mapped to Specific Kill Chains
Active Sessions		20 detected active sessions	Zero active sessions spotted	Zero active sessions spotted	12 active sessions spotted
Identified Vulnerabilities and Exposures		High and low Value (DCSync, RBCD, AS-REP Roasting, Kerberoasting, RDP/SMB)	RDP/SMB and Kerberoasting DCSync Rights and AS-REP Roasting	RDP/SMB and Kerberoasting, DCSync Rights and AS-REP Roasting	All High-Value (DCSync, RBCD, AS-REP Roasting, Kerberoasting, RDP/SMB)
Affected Assets		One (*DC01.PHANTOM.CORP*)	One (*KINGSLANDING.SEVENKINGDOMS.LOCAL*)	One (*WINTERFELL.NORTH.SEVENKINGDOMS.LOCAL*)	Two (*WINSERVER.INTERTECH.LOCAL, CLIENT2.INTERTECH.LOCAL*)
Number of Identified Lateral Movement Paths		21	1	2	12
Number of Identified Privilege Escalation Paths		15	4	6	128
Impact	Account Takeover	3	1	4	32
	Credential Theft	20	2	2	12
	Direct DC Access	Zero	Zero	Zero	8
	Full AD Domain Compromise	4 paths	3 paths	2 paths	51 paths

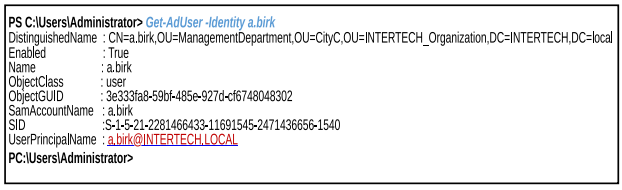


Fig. 9. Active Directory enumeration revealing the object properties and organizational unit (OU) placement of a.birk.



Fig. 10. Successful LDAP authentication using NetExec, confirming valid credentials and network reachability.

Identity Enumeration and Hierarchy Mapping

The validation process began by establishing the target account's properties and its position within the logical AD hierarchy. Utilizing the Get-ADUser cmdlet, we extracted the account's Distinguished Name (DN) to determine its organizational context as shown in Fig. 9.

As illustrated in Fig. 9, a.birk is located within the Management Department OU. Under a standard security posture, this placement implies the account should be restricted to localized, non-administrative permissions. However, the detection engine flagged this account due to its relationship with sensitive security descriptors. Following identity verification, we simulated the perspective of an adversary to confirm the usability of the credentials across the network. The NetExec (NXC) framework was utilized to perform a cross-protocol authentication audit against the Domain Controller (DC) located at 192.168.56.113.

Fig. 10 displays a successful LDAP authentication (indicated by the [+] status). This validates that the credentials a.birk:Password@123 are functionally active and that the DC's LDAP interface is accessible for remote enumeration and attribute modification requests.

Exploitation of Access Control List (ACL) Misconfigurations The core vulnerability identified by our engine involves a "security debt" within the domain's permission architecture. Automated pathfinding and manual Access Control List (ACL) inspection revealed that users in the Management Department had been inadvertently granted GenericWrite or WriteProperty permissions over the CityC IT Admins security group. In this organizational architecture, membership in CityC IT Admins is functionally equivalent to Domain Admin privileges due to group nesting. By leveraging this ACL weakness, an attacker could programmatically add a new principal or elevate the a.birk account into the CityC IT Admins group. This operation utilizes the Security Account Manager Remote (SAMR) or LDAP protocols to update the member attribute of the privileged group.

Forensic Artifact Validation and Detection We validated the final stage of this case study by analyzing the forensic artifacts generated during the escalation with detail finding listed in Table 4. Upon the simulated modification of the group membership, the security logs

Table 4
Observed Security Events during Validations.

Event ID	Category	Description
4728	Member Added	A member was added to a security-enabled global group.
4624	Account Logon	Successful logon of a.birk preceding the modification.
4672	Special Privileges	Assigned to the new session if administrative rights were exercised.

on the Domain Controller were audited to confirm that the detection engine could correlate the activity with known attack patterns.

This case study confirms that the detection engine accurately identifies not only the presence of compromised credentials but also the high-risk architectural flaws that allow those credentials to be used for full domain compromise.

4.6. Empirical validation of the ILP optimizer

To validate the ILP engine, a complete remediation lifecycle was executed against the INTERTECH.LOCAL domain. The initial enumeration identified a highly vulnerable baseline comprising $|\mathcal{P}| = 170$ distinct multi-hop attack paths, driven largely by 128 privilege escalation vectors and unconstrained delegation misconfigurations. Table 5 outlines the details of these findings.

A conventional heuristic approach applied to this environment would prompt sweeping ACL resets or the blanket revocation of administrative memberships, directly risking a Denial-of-Service (DoS) condition. Conversely, the VACIR ILP engine formulated a constrained hitting set, determining that the entire attack surface could be neutralized through exactly 56 targeted interventions (Objective Cost: 105.0).

Crucially, this execution validated the operational-safety constraint logic. The ILP was programmed with a strict protection constraint ($x_a = 0$) for the built-in ADMINISTRATOR account. While a heuristic model would classify this account as a primary target for privilege stripping due to its DOMAIN ADMINS membership, the VACIR solver successfully preserved it. The optimizer mathematically routed the remediation strategy around the constraint, surgically severing the converging lateral movement pathways feeding into the group without altering the protected principal.

To empirically validate the ILP optimizer's structural impact, direct PowerShell telemetry was captured from the INTERTECH.LOCAL domain controller (Fig. 11 and Fig. 12 illustrate the system state prior to remediation, while Fig. 13 and Fig. 14 show the outcome after the remediation plan was implemented). Initial baseline queries revealed severe operational over-privilege. The graph contained 51 direct and nested accounts holding DOMAIN ADMINS rights, effectively elevating hundreds of standard users across the Management, HR, and Financial departments due to recursive Access Control List (ACL) misconfigurations.

Using the ILP derived minimum cost hitting set, the orchestrator precisely cut those exploitable links. Post-remediation telemetry verifies that administrative boundaries are now strictly enforced. The optimizer reduced DOMAIN ADMINS membership from 51 down to exactly 2 authorized principals. Importantly, the action safely disentangled the remaining users, returning formerly exposed accounts to their intended unprivileged Organizational Units while preserving operational safety and avoiding any service disruption.

After remediation and ILP have been implemented, VACIR extend Time-to-Compromise (TTC) from minutes to mathematically unreachable.

```
PS C:\Users\Administrator> $da=(Get-ADGroupMember "Domain Admins" -Recursive).SID.Value; Get-ADUser -Filter * -Properties CanonicalName | Select-Object @{N='Name';E={if($da -contains $_.SID.Value){'DomainAdmins'}else{$_.CanonicalName.Split('/')[-2]}}} | Group-Object Name | Select-Object Name, Count | Sort-Object Count -Descending
Name                    Count
----                    -----
Management Department   100
HR Department           100
Financial Department    100
DomainAdmins            51
NormalUser              50
IT Department           21
Users                   3
```

Fig. 11. Pre Remediation Recursive Membership Analysis. Deep-level group extraction demonstrating structural over privilege and security debt. Due to recursive ACL misconfigurations and nested group memberships, hundreds of standard users across the Management, HR, and Financial departments inadvertently inherited domain-level administrative execution rights.

```
PS C:\Users\Administrator> [PSCustomObject]@{ Name = "DomainAdmins"; Count = (Get-ADGroupMember -Identity "Domain Admins").Count }
Name          Count
----          -----
DomainAdmins  51
```

Fig. 12. Baseline Enumeration of Administrative Privilege. Direct PowerShell telemetry querying the DOMAIN ADMINS security group prior to ILP optimization, revealing a highly vulnerable baseline state of 51 elevated accounts.

```
PS C:\WINDOWS\system32> $da=(Get-ADGroupMember "Domain Admins" -Recursive).SID.Value; Get-ADUser -Filter * -Properties CanonicalName | Select-Object @{N='Name';E={if($da -contains $_.SID.Value){'DomainAdmins'}else{$_.CanonicalName.Split('/')[-2]}}} | Group-Object Name | Select-Object Name, Count | Sort-Object Count -Descending
Name          Count
----          -----
NormalUser    75
ServiceUsers  25
DomainAdmins  2
Users         2
```

Fig. 13. Post-Remediation Recursive Membership and Privilege Decoupling. Post-optimization group extraction illustrating the successful enforcement of least-privilege boundaries. The ILP engine surgically decoupled the previously exposed unprivileged organizational units, returning standard users and service accounts to their designated, non-administrative security contexts without violating operational-safety constraints.

```
PS C:\WINDOWS\system32> [PSCustomObject]@{ Name = "DomainAdmins"; Count = (Get-ADGroupMember -Identity "Domain Admins").Count }
Name          Count
----          -----
DomainAdmins  2
```

Fig. 14. Post-Remediation Validation of the ILP Optimizer. Post-execution telemetry confirming the solver's precise structural impact. The minimum-cost hitting set successfully severed all unauthorized attack edges, strictly reducing the DOMAIN ADMINS membership from 51 to exactly 2 authorized principals.

5. Discussion

5.1. Advantages to researchers and practitioners

Formal Graph-Based Security Modeling VACIR introduces a rigorous graph-theoretic representation of AD environments, where users,

Table 5
Remediation impact and security metrics.

Security metric	Initial state (S_0)	Post remediation state (S_1)	Mitigation impact
Total Viable Attack Paths	170	2	98.8% Reduction
Lateral Movement Paths	12	0	100.0% Mitigated
Over Privileged Escalation Paths	128	0	100.0% Mitigated
Required Remediation Actions ($\|\mathcal{A}_{opt}\|$)	170 (Heuristic 1:1 Mitigation)	56 (ILP Optimized)	67.1% Operational Overhead Reduction (Minimum-Cost Hitting Set)
Protected Built-in Admin	Vulnerable to Pivoting	Safely Isolated	Zero Business Disruption
Execution Time (seconds)	0.29	14.39	Practical Runtime Scaling

computers, groups, and organizational units are modeled as interconnected nodes and relationships. This provides researchers with a mathematically structured framework for studying attack-path discovery, privilege escalation, and lateral movement in enterprise networks.

Reproducible Experimental Environment The framework is validated within a controlled cyber range using Infrastructure-as-Code (IaC), automated provisioning, and repeatable snapshot-based experimentation. This allows future studies to reproduce experiments consistently and evaluate new defensive strategies under standardized conditions.

Support for Realistic Adversary Modeling VACIR models real-world attack techniques including Kerberoasting, AS-REP Roasting, DCSync, RDP abuse, ACL misconfigurations, and lateral movement. This enables researchers to evaluate attack-path analysis methods against realistic enterprise threat scenarios rather than simplified simulations.

Accurate Identification of Critical Attack Paths VACIR enables security administrators to identify exploitable attack paths, privilege escalation routes, and lateral movement opportunities within AD environments with high precision and minimal false positives.

Practical Support for Defensive Decision-Making VACIR assists security teams in prioritizing vulnerabilities and remediation actions based on attack feasibility, exploit cost, and operational impact, thereby improving strategic cybersecurity decision-making.

5.2. Limitation and future work

A. Limitations Ground-truth and metric sensitivity.

Our evaluation relies on ground-truth labels (GT) to compute TP/FP/FN and derived metrics. However, exploitability in AD is often context-dependent (e.g., account state, segmentation, or service dependencies). This introduces a threat to validity: if GT labeling does not fully capture environmental constraints, then confusion-matrix metrics may over- or under-estimate true operational exploitability.

Dataset and environment representativeness.

The laboratory domain is implemented in an isolated VirtualBox cyber range (host-only network) with a limited number of hosts (DC + three Windows clients + attacker VM) and a structured OU design (cities/departments). While this controlled environment supports repeatability, it may not reflect the complexity of production environments (e.g., large forests, complex trust relationships, hybrid identity). The proposal document also notes the importance of validating scalability on larger graphs; the current evaluation includes public datasets and a lab domain but does not yet provide a dedicated scaling study.

Scalability and Performance on Large Synthetic Graphs. To evaluate the algorithmic scalability of the VACIR framework on larger enterprise architectures, future benchmarking will incorporate synthetically generated Active Directory attack graphs scaling to tens of thousands of nodes, aligning with the graph sizes evaluated in prior optimization studies (Guo et al., 2023). While VACIR currently processes the `INTERTECH.local` domain in 0.29 s (Table 2), analyzing the (ILP) solver's performance under increasingly dense ACL matrices and large-scale multi-hop paths will provide further empirical validation regarding solver efficiency and memory scaling in massive enterprise deployments.

Data acquisition assumptions and toolchain dependency.

The dataset is built from an adversary-emulation workflow using tools such as Nmap, Responder, Hashcat, Evil-WinRM, and SharpHound. This provides an adversary-centric view but also means that the graph fidelity depends on the privileges obtained and the visibility SharpHound can achieve from that context. Any gaps in enumeration or session visibility can translate into missed edges in G, thereby affecting recall and downstream remediation decisions.

B. Future Work

The results presented in this paper establish a foundation for optimization-driven AD remediation. However, several open challenges and promising research directions remain. This section outlines five avenues for extending the scope, rigor, and applicability of the VACIR framework.

Hybrid and Cloud-Extended Identity Environments.

Modern enterprise identity architectures increasingly span on-premises AD, Azure AD (now Entra ID), and federated identity providers. In such hybrid environments, attack paths are no longer confined to a single on-premises domain; adversaries can traverse cloud–on-premises trust boundaries through mechanisms such as Azure AD Connect synchronization abuse, OAuth token manipulation, or cross-tenant privilege escalation. The current version of VACIR models only on-premises AD relationships extracted via SharpHound. Extending the security state graph G to incorporate hybrid identity primitives—including cloud-side group memberships, application registrations, conditional access policies, and cross-environment trust edges—would enable VACIR to reason about end-to-end attack chains that span both cloud and on-premises boundaries. This extension is increasingly important as organizations adopt hybrid identity models as their default architecture.

Integration of Host-Level and Network-Level Context.

VACIR currently operates exclusively on the AD relationship graph extracted via SharpHound, which captures directory-level objects, group memberships, ACLs, sessions, and delegation settings. However, practical exploitability of a given attack path also depends on factors not visible in the directory graph alone. These include host-level firewall rules that may block lateral movement traffic, endpoint detection and response (EDR) policies that can intercept credential-based attacks, network segmentation boundaries that restrict inter-VLAN communication, and account lockout thresholds that limit brute-force or password-spraying attempts. Incorporating lightweight telemetry

from these sources into the cost function W and the constraint model would improve the fidelity of exploitability assessments. Specifically, edges corresponding to paths blocked by network controls or monitored by EDR agents could receive higher cost weights, thereby reducing false negatives (missed paths that are genuinely blocked) and potentially increasing recall without degrading precision. This integration represents a meaningful step toward bridging the gap between graph-theoretical reachability and practical, real-world exploitation feasibility.

Temporal and Continuous Monitoring.

The current VACIR pipeline operates on a single-snapshot dataset collected at a specific point in time. In operational settings, AD configurations evolve continuously as administrators modify group memberships, provision and deprovision accounts, adjust delegation settings, and update group policies. A static, point-in-time analysis may therefore become stale rapidly, particularly in dynamic environments with frequent administrative changes. Future work will explore incremental graph update mechanisms that allow VACIR to re-evaluate remediation recommendations as new SharpHound collections arrive, without requiring full re-ingestion and re-optimization of the entire graph. This would transform VACIR from a one-time assessment tool into a continuous monitoring and hardening workflow, enabling defenders to detect and respond to newly introduced misconfigurations or attack-path changes in near real time. Such a capability would align VACIR with the operational cadence of modern security operations centers (SOCs), where continuous posture management is increasingly expected.

6. Conclusion

Enterprise AD environments remain highly susceptible to lateral movement and privilege escalation driven by misconfigurations and vulnerability-backed attack primitives. Although existing tools effectively reveal attack paths, they do not provide a principled mechanism for selecting a minimal and operationally safe remediation strategy. This paper introduced VACIR, a vulnerability-aware remediation co-pilot that transforms adversary-collected AD relationship data into a weighted security state graph and applies formal optimization (ILP/SMT) to compute prioritized remediation actions aimed at disrupting high-value attack paths while minimizing operational impact. Empirically, VACIR demonstrates strong and consistent detection correctness across one laboratory domain and three public datasets, achieving Precision = 1.00 and FPR = 0.00 in all evaluated cases, with sub-second runtime (less than or equal to 0.29 s). Compared to conventional graph-analysis baselines, VACIR provides a substantially improved trade-off between correctness and computational cost, supporting interactive and iterative what-if remediation analysis. These results provide evidence that integrating vulnerability-aware modeling with optimization is a promising direction for closing the longstanding gap between attack-path discovery and actionable, low-disruption remediation in AD defense.

CRediT authorship contribution statement

Tegha Hamza Akwo: Writing – original draft, Validation, Software, Methodology, Investigation. **Dekai Zhang:** Resources. **Lu Yu:** Resources. **Bing Zhang:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Natural Science Foundation of Hebei Province under Grant F2026203059, S&T Program of Hebei under Grant 264Z0701G, Basic Innovation Research Cultivation Program of Yanshan University under Grant 2024LGZD004, and National Natural Science Foundation of China under Grant 62376240. The authors are grateful to the valuable comments and suggestions of the reviewers.

Data availability

The source code, detection engine, and raw AD telemetry datasets generated during this study are openly available in the VACIR repository at https://github.com/TeghaHamza/VACIR-Engine-Code-/releases/tag/V1.0.

References

Carson, J., 2022. Privilege escalation on windows: When you want it and when you don't. [Online]. Available: URL https://delinea.com/blog/windows-privilege-escalation#what-is-attack. Delinea Blog.

Cordis, G.A., Costea, F.M., Pecherle, G., Győrödi, R., Győrödi, C., 2023. Considerations in mitigating kerberos vulnerabilities for active directory. In: Proceedings of the 2023 17th International Conference on Engineering of Modern Electric Systems. EMES, Oradea, Romania.

cyber.gov.au/, 2024. Detecting and mitigating Active Directory compromises. [Online]. Available: URL https://www.cyber.gov.au/business-government/detecting-responding-to-threats/detecting-and-mitigating-active-directory-compromises.

Dewri, R., Poolsappasit, N., Ray, I., Whitley, L.D., 2007. Optimal Security Hardening Using Multi-objective Optimization on Attack Tree Models of Networks. [Online]. Available: URL https://du-researchportal.esploro.exlibrisgroup.com/esploro/outputs/conferenceProceeding/Optimal-security-hardening-using-multi-objective-optimization/991042206876002766?utm_source=.

Dunagan, J., Zheng, A., Simon, D.R., 2009. Heat-Ray: Combating Identity Snowball Attacks Using Machine Learning, Combinatorial Optimization and Attack Graphs. [Online]. Available: URL https://www.microsoft.com/en-us/research/publication/heat-ray-combating-identity-snowball-attacks-using-machine-learning-combinatorial-optimization-and-attack-graphs/?utm_source.

Felicite, J.G.V., et al., 2024. Detecting pass-the-hash attack in a microsoft active directory environment using an open-source approach. In: 2024 12th International Conference in Software Engineering Research and Innovation. CONISOFT, Puerto Escondido, Mexico, Author list truncated in source.

Felicite, J.G.V., et al., 2026. An open-source approach to detect pass-the-hash attack in active directory using wazuh and sysmon.

Goel, D., Ward-Graham, M.H., Neumann, A., Neumann, F., Nguyen, H., Guo, M., 2022. Defending Active Directory by Combining Neural Network Based Dynamic Program and Evolutionary Diversity Optimisation. [Online]. Available: URL https://cs.adelaide.edu.au/~optlog/research/cfamd.php?utm_source.

Guo, M., Ward, M., Neumann, A., Neumann, F., Nguyen, H., 2023. Scalable Edge Blocking Algorithms for Defending Active Directory Style Attack Graphs, " AAAI 2023, pp. 5649–5656. [Online]. Available: URL https://ojs.aaai.org/index.php/AAAI/article/view/25701?utm_source.

Guo, M., et al., 2022. Practical fixed-parameter algorithms for defending active directory style attack graphs. In: Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence. AAAI-22.

Herranz-Oliveros, D., et al., 2024. Unsupervised learning for lateral-movement-based threat mitigation in active directory attack graphs. Electronics 13 (19).

Horizon3, 2025. How Attackers Own Active Directory in Minutes. [Online]. Available: URL https://horizon3.ai/attack-research/attack-blogs/the-quiet-attack-path/.

Kapko, M., 2026. CrowdStrike annual global threat report: Attack breakout time. [Online]. Available: URL https://cyberscoop.com/crowdstrike-annual-global-threat-report-attack-breakout-time/. CyberScoop.

Kohler, J., 2022. Getting started. [Online]. Available: URL https://www.cyberdefensemagazine.com/getting-started/. Cyber Defense Magazine.

m4lwhere, 2024a. Bloodhound-CE-sample-data (north) — GitHub repository. [Online]. Available: URL https://github.com/m4lwhere/Bloodhound-CE-Sample-Data/blob/main/NORTH_20240410083414_BloodHound-2.3.3.zip. (Accessed May 2024).

m4lwhere, 2024b. Bloodhound-CE-sample-data (seven kingdoms) — GitHub repository. [Online]. Available: URL https://github.com/m4lwhere/Bloodhound-CE-Sample-Data/blob/main/SEVENKINGDOMS_20240410083609_BloodHound-2.3.3.zip. (Accessed May 2024).

Meghana, K., Shankaramma, Thippeswamy, M., Anu, A.M., Chaithra, M., Nagamani, N., 2025. Enumeration and post-enumeration attack on active directory and their detection using log correlation method. In: 2024 8th International Conference on Computational System and Information Technology for Sustainable Solutions. CSITSS, Bengaluru, India.

Meghana, K., et al., 2024. Enumeration and post-enumeration attack on active directory and their detection using log correlation method. In: 2024 8th International Conference on Computational System and Information Technology for Sustainable Solutions. CSITSS, Bengaluru, India, Variant of [23].

Microsoft, 2025. Microsoft sentinel. [Online]. Available: URL https://www.microsoft.com/en-us/security/business/siem-and-xdr/microsoft-sentinel. (Accessed September 2025).

Microsoft, 2026. Microsoft defender for identity: Security posture assessments. [Online]. Available: URL https://learn.microsoft.com/en-us/defender-for-identity/security-posture-assessments/accounts.

Microsoft Corporation, 2025. Active directory domain services overview. [Online]. Available: URL https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview. Online.

Moric, Z., Dakic, V., Redzepagic, J., Otocan, F., 2024. Penetration testing and exploitation of active directory configuration vulnerabilities. In: 2024 4th Intelligent Cybersecurity Conference. Valencia, Spain.

Nguyen, A.C., Nguyen, H., 2023. Cozure: Context free grammar co-pilot tool for finding new lateral movements in azure active directory. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses. RAID'23, Hong Kong, China.

Nguyen, G.-H., et al., 2025. Automated framework for active directory security and compliance through continuous monitoring and attack path validation. In: 2025 9th International Conference on Computational System and Information Technology for Sustainable Solutions. CSITSS, Nan, Thailand.

Noel, S., Jajodia, S., O'Berry, B., Jacobs, M., 2003. Efficient Minimum-Cost Network Hardening via Exploit Dependency Graphs. [Online]. Available: URL https://csis.gmu.edu/noel/pubs/2003_ACSAC.pdf?utm_source.

Quest, 2026. What is azure active directory? [Online]. Available: URL https://www.quest.com/learn/what-is-azure-active-directory.aspx. Online.

SpecterOps, 2025a. What is Attack Path Management? [Online]. Available: URL https://specterops.io/what-is-attack-path-management/?utm_source=#what-are-attack-paths.

SpecterOps, 2025a. Introduction to BloodHound. [Online]. Available: URL https://bloodhound.specterops.io/get-started/introduction. (Accessed September 2025).

SpecterOps, 2025b. BloodHound Community Edition — Ingest Sample Data Quickstart. [Online]. Available: URL https://bloodhound.specterops.io/get-started/quickstart/ce-ingest-sample-data.

SpecterOps, 2025b. SharpHound — GitHub repository. [Online]. Available: URL https://github.com/SpecterOps/SharpHound. (Accessed June 2025).

Splunk, 2025. Splunk (home page). [Online]. Available: URL https://www.splunk.com/. Online. (Accessed August 2025).

Tenable, 2026. Mean time to remediate (MTTR). [Online]. Available: URL https://www.tenable.com/cybersecurity-guide/learn/mean-time-to-remediate-mttr.

Wang, L., Noel, S., Jajodia, S., 2006. Minimum-Cost Network Hardening Using Attack Graphs. [Online]. Available: URL https://www.sciencedirect.com/science/article/abs/pii/S0140366406002271?utm_source.

WindowsForum, 2025. Critical CVE-2025–21293 vulnerability in active directory: What you need to know. [Online]. Available: URL https://windowsforum.com/threads/critical-cve-2025-21293-vulnerability-in-active-directory-what-you-need-to-know.349506/.

Zhou, J., et al., 2024. Lateral movement detection via time-aware subgraph classification on authentication logs. Author list incomplete in source, Hangzhou, China.